

Chaos labs

Theory, evidence and controlled experiments

Read the relevant theory, predict, measure, explain, and check recovery. Each lab tests one question. The solutions are collected at the end so you can study and run the question without seeing its expected observations.

Run `./lab.sh` provision once. Choose any lab: `./lab.sh NN` setup prepares and verifies its fixture in the shared VM. Open `./lab.sh ssh`, run the procedure and record your evidence. Compare NN solution. Save results before NN reset removes that lab's resources and files.

Stop if the baseline fails. Confirm the fault actually reached its target. Expected results are not your observations. Use Bash without `set -e` and preserve the same shell when blocks share variables. Keep a second VM terminal available for recovery.

All labs use one VM. Each Kubernetes lab owns a separate named kind cluster inside it. No lab sequence is required. Reset finished labs to free resources. `./lab.sh stop` halts the whole VM while preserving files. See README.md for capacity.

Long command lines wrap visually in this print edition. Copy exact commands from docs/chaos-labs.md or the terminal question card. Theory citations use original section numbers in docs/chaos-theory.md.

Lab	Topic	Lab	Topic
00	Prepare the learning environment	11	Health depends on what you test
01	A timeout makes a silent failure manageable	12	Locate startup failures and measure recovery
02	Prove why a process was killed	13	Placement, supervision and replacement
03	Restart policies have limits	14	Quorum, convergence and application availability
04	CPU contention and a quota	15	Count how layered retries multiply work
05	A container is several Linux mechanisms	16	Design one clear experiment
06	Separate containers can share a full filesystem	17	Signal delivery and the shutdown budget
07	Network delay accumulates across sequential calls	18	Persistence, ownership and read-only mounts
08	A syscall error tests application handling	19	Unscheduled Pods versus memory-killed containers
09	A syscall denial need not crash the process	20	Trace DNS, Service ports and the application listener
10	Ownership, disruption budgets and Service routing	21	Configuration freshness and a stalled rollout

Lab 00 - Prepare the learning environment

Which checks establish a usable baseline before you inject a fault?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 00 setup` on the host. No other lab is required.

Theory you need

A chaos experiment compares a measured outcome before, during and after one controlled fault. Observability means you can measure that outcome; a baseline is its normal value or range. If the no-fault check fails, stop: a later failure would not tell you whether the injection caused it.

The method is: choose a measurement, establish normal behaviour, predict a result under a specified fault, run the comparison, and explain the evidence. For example, “the client still responds within one second when its cache is unavailable” is testable; “the service is resilient” is not.

Blast radius is everything the experiment can affect. These labs target named processes, containers or objects inside a disposable Linux VM. Namespaces separate resource views; cgroup v2 accounts for and limits resource use. The VM still shares CPU, memory and a kernel across labs.

Record versions so another run can be compared with yours. Keep evidence in this lab's workspace. Reset deletes that workspace; shared tooling and other labs remain available. Each later experiment must still measure its own service baseline and recovery.

Source: docs/chaos-theory.md: §§1.3, 2.1, 2.5; Appendix A.

Experiment

Check the shared Ubuntu VM and save a baseline record inside Lab 00. No fault is injected.

Before running: If a prerequisite already fails, can a later failure tell you anything about the injected fault?

Measurement key

Signal	Meaning
<code>cgroup2fs / cgroups=2</code>	The kernel and Docker use the cgroup v2 interface used by these labs.
<code>hello-world</code>	A successful run checks image access, container creation and process execution together.
<code>df -h /</code>	Size is total filesystem capacity; Avail is unused space. The shared provisioner requires at least 60 GiB total.
<code>versions / baseline.txt</code>	Record the actual versions and baseline checks, not simply “setup passed”.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 00 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Check the kernel, cgroup, Docker and disk baseline

Run in: VM terminal 1

```
cd ~/labs/lab00
set -o pipefail
{
  uname -srmo
  stat -fc %T /sys/fs/cgroup
  docker info --format 'driver={{.CgroupDriver}} cgroups={{.CgroupVersion}}'
  docker run --rm --name ce-lab00-check hello-world
  df -h /
} 2>&1 | tee checks.txt
```

Record: Copy each observed value into the check table. Mark any failed command or missing prerequisite as action needed before continuing; checks.txt preserves the terminal output.

Step 2. Record the source commit, versions and image digests

Run in: VM terminal 1

```
cd ~/labs/lab00
{
  cat ~/labs/book-commit.txt ~/labs/versions.txt
  for image in python:3.12-slim ubuntu:24.04 busybox:1.36 nginx:1.27 ghcr.io/shopify/toxiproxy:2.12.0
    bloomberg/goldpinger:3.11.3; do
    printf '%s ' "$image"
    docker image inspect "$image" --format 'id={{.Id}} digests={{json .RepoDigests}}'
  done
} 2>&1 | tee versions-images.txt
```

Record: Save the source commit, tool versions, and each image tag, local ID and registry digest from versions-images.txt. These identify the environment used for the baseline.

Step 3. Save the baseline record

Run in: VM terminal 1

```
cd ~/labs/lab00
{ date --iso-8601=seconds; cat checks.txt versions-images.txt; } > baseline.txt
cat baseline.txt
```

Record: Note ~/labs/lab00/baseline.txt as the evidence location and confirm it includes the hello-world result as well as the versions. Preserve a copy before reset if you need it later.

Recovery check: Prerequisite checks pass and baseline.txt records them.

Write your answer

Use the observations recorded beside each step.

Check	Observed value	Pass or action needed
cgroup / Docker		
Container execution		
Disk capacity		
Versions / baseline record		

1. Which kernel, cgroup, Docker, container-execution and disk checks passed? Give their observed values.

2. Which source commit, tool versions and image identities did you save, and where is the baseline record?

3. Why must you fix a failed baseline before injecting a fault?

Apply the same reasoning

Why must Lab 01 check Redis again even when hello-world succeeds?

Save results before removing this lab's resources and files: `./lab.sh 00 reset`

Lab 01 - A timeout makes a silent failure manageable

Why does the exception handler run after refusal but not while packets are silently dropped?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 01 setup` on the host. No other lab is required.

Theory you need

A dependency call must return or raise an error before execution can reach the next statement or an exception handler. `try/except` supplies a reaction to an error; it does not give the call a deadline.

REJECT with a TCP reset gives the client a prompt refusal. DROP silently discards matching packets. Without a reply the network stack retries and waits; its eventual failure can be much later than a useful application deadline. These are two different failure modes of the same dependency.

A connect timeout bounds connection establishment. A read timeout bounds waiting for data after connecting. The prepared client disables retries and sets both to TIMEOUT when supplied. A socket timeout is not a general end-to-end deadline across many operations or retries.

The five-second timeout command is an external watchdog: it terminates the test process. It does not execute the client's fallback. Compare that termination with the client catching its own timeout, then remove the firewall rule and check the same request again.

Source: docs/chaos-theory.md: §1.5 and Experiment Card 1.1.

Experiment

One `ce-lab01` firewall rule targets TCP 127.0.0.1:6381. All fault runs use a five-second watchdog; `TIMEOUT=0.5` sets the client timeouts. The comparison runs in `ce-lab01-runner.service` so `reset` can stop it before removing its firewall rules.

Before running: Predict which of the three attempts reaches the handler: REJECT, DROP, and DROP with a 0.5-second timeout.

Measurement key

Signal	Meaning
OK / DEGRADED	OK means Redis answered. DEGRADED means the client caught a Redis error and executed its fallback.
elapsed	Seconds measured inside the client. A watchdog-killed, buffered process may print no elapsed line.
exit=124	The external timeout command reached its deadline. This is not the application's handled error.
iptables pkts	A nonzero counter confirms traffic matched the fault rule; no matches means you have not tested that fault.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 01 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Check the Redis baseline

Run in: VM terminal 1

```
cd ~/labs/lab01
redis-cli -p 6381 ping
env -u TIMEOUT timeout 5s python3 client.py
echo "client exit=$?"
```

Record: Fill the Baseline row with OK or error, elapsed time and client exit status. OK means the request succeeded; the fallback handler was not needed.

Step 2. Case 1 - REJECT without an application timeout

Run in: VM terminal 1

```
cd ~/labs/lab01
sudo systemd-run --unit=ce-lab01-runner --collect --wait --pipe \
  --uid="$(id -u)" --property=RuntimeMaxSec=20s --working-directory="$PWD" \
  /bin/bash "$PWD/run.sh" reject
```

Record: Fill the REJECT row with any handler message, elapsed time, client exit and matched packets. Use the output to identify what ended the wait.

Step 3. Case 2 - DROP without an application timeout

Run in: VM terminal 1

```
cd ~/labs/lab01
sudo systemd-run --unit=ce-lab01-runner --collect --wait --pipe \
  --uid="$(id -u)" --property=RuntimeMaxSec=20s --working-directory="$PWD" \
  /bin/bash "$PWD/run.sh" drop
```

Record: Fill the DROP row with whether a handler or elapsed line appeared, client exit, whole seconds waited and matched packets. Identify what ended the wait from those observations.

Step 4. Case 3 - DROP with a 0.5-second application timeout

Run in: VM terminal 1

```
cd ~/labs/lab01
sudo systemd-run --unit=ce-lab01-runner --collect --wait --pipe \
  --uid="$(id -u)" --property=RuntimeMaxSec=20s --working-directory="$PWD" \
  /bin/bash "$PWD/run.sh" drop-timeout
```

Record: Fill the DROP + 0.5 s timeout row with the handler message, elapsed time, client exit and matched packets. Compare what ended this wait with Case 2.

Step 5. Prove Redis and firewall recovery

Run in: VM terminal 1

```
cd ~/labs/lab01
redis-cli -p 6381 ping
env -u TIMEOUT timeout 5s python3 client.py
echo "client exit=$?"
if rules=$(sudo iptables -S OUTPUT); then
  printf '%s\n' "$rules" | grep -- '--comment ce-lab01'
  echo "grep exit=$? (1 means no lab rule remains)"
else
  echo 'UNKNOWN: firewall rules could not be read; fault removal is not verified.'
fi
```

Record: Fill the Recovered row with the response, elapsed time and exit status. Save the firewall search output and status to establish whether the fault rule remains.

Recovery check: The client succeeds again and no ce-lab01 rule remains.

Only if normal recovery fails

Step 1. Manual rule removal, only if run.sh was killed

Run in: VM terminal 2

```
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j DROP
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j REJECT --reject-with
tcp-reset
env -u TIMEOUT timeout 5s python3 ~/labs/lab01/client.py
echo "client exit=$?"
```

Write your answer

Use the observations recorded beside each step.

Attempt	Handler ran?	Elapsed / exit	Rule packets	What ended the wait?
Baseline				
REJECT				
DROP				
DROP + 0.5 s timeout				
Recovered				

1. Did the baseline and recovered requests succeed? What proves the firewall fault was removed?
2. For REJECT, DROP and DROP with TIMEOUT=0.5, did the handler run, what ended the wait, and what were the elapsed time and exit status? Cite each rule's packet counter.
3. Why does try/except need an application timeout to handle a silent dependency promptly?

Apply the same reasoning

If each attempt has a 0.5-second timeout and you allow three attempts, is the whole request bounded by 0.5 seconds?

Save results before removing this lab's resources and files: `./lab.sh 01 reset`

Lab 02 - Prove why a process was killed

Why is exit status 137 insufficient to diagnose an OOM kill?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 02 setup` on the host. No other lab is required.

Theory you need

A signal is a notification sent to a process. SIGTERM (15) can be handled for cleanup; SIGKILL (9) cannot be caught. Bash commonly reports signal termination as 128 + signal number, so a SIGKILL gives 137. A program may also explicitly exit with 137: status alone is not a diagnosis.

The kernel can invoke the out-of-memory (OOM) killer when it cannot satisfy memory demand. A cgroup has its own memory boundary: its processes can hit MemoryMax even while the rest of the VM has available memory. The allocator here repeatedly creates and touches bytes, so it demands real memory rather than merely reserving virtual addresses.

MemoryMax=128M and MemorySwapMax=0 constrain this experiment. RuntimeMaxSec=20s is a separate time bound. An OOM kill, that timer, and a deliberate SIGKILL can all stop the process; identify which happened by matching the unit, PID and time in the logs.

The systemd-run command reports the transient unit's outcome; its shell status need not equal the child's Bash status. Collect the unit result and kernel evidence. systemd-oomd is a userspace memory-pressure killer, so an oomd action is a different cause from a kernel OOM kill.

Source: docs/chaos-theory.md: §2.3 (exit codes, signals and OOM), §5.5.2 (memory limits).

Experiment

The allocator runs only in ce-lab02, with 128 MiB memory, no swap and a 20-second runtime bound.

Before running: Will a deliberate SIGKILL and a memory-limit kill have different exit statuses? Which evidence can distinguish them?

Measurement key

Signal	Meaning
deliberate KILL exit	Read \$? immediately after the process. An intervening command would replace it.
journalctl -u ce-lab02	Look for the recorded termination signal and result: oom-kill and timeout describe different causes.
journalctl -k	Match a memory-cgroup OOM / Killed process line to this allocator and experiment time.
journalctl -u systemd-oomd	A matching userspace-kill entry changes the diagnosis; unrelated historical entries do not.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 02 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Confirm memory controls and begin an evidence record

Run in: VM terminal 1

```
cd ~/labs/lab02
grep -w memory /sys/fs/cgroup/cgroup.controllers
systemctl is-active ce-lab02.service; echo "unit status=$? (nonzero means not active)"
```

Record: Record the controller listing and named unit state before the comparison. Stop if the memory controller is missing or a previous allocator is active.

Step 2. Record a deliberate SIGKILL

Run in: VM terminal 1

```
python3 -c 'import os,signal; os.kill(os.getpid(),signal.SIGKILL)'
echo "deliberate KILL exit=$?"
```

Record: Fill the Known SIGKILL row with the immediately printed exit status and the explicit self-sent signal as the cause. Keep these observations for comparison with the allocator.

Step 3. Run the allocator within its memory and time limits

Run in: VM terminal 1

```
cd ~/labs/lab02
date --iso-8601=seconds > started-at.txt
sudo systemd-run --unit=ce-lab02 --wait --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p RuntimeMaxSec=20s \
  /usr/bin/python3 -c 'import os
print("allocator PID=" + str(os.getpid()), flush=True)
a=[]
while True: a.append(b"x"*1000000)'
echo "systemd-run exit=$? (unit outcome, not the child's Bash status)"
date --iso-8601=seconds > ended-at.txt
```

Record: Save the printed systemd-run status and the start/end timestamps. The allocator PID is written to the unit journal, which you will read next.

Step 4. Match the allocator termination to unit, kernel and userspace logs

Run in: VM terminal 1

```
cd ~/labs/lab02
since=$(cat started-at.txt)
cat started-at.txt ended-at.txt
sudo journalctl -u ce-lab02 --since "$since" --no-pager | tee unit.log
sudo journalctl -k --since "$since" --no-pager | grep -iE 'oom|killed process' | tee kernel.log
sudo journalctl -u systemd-oomd --since "$since" --no-pager | tee oomd.log
```

Record: Fill the allocator row with its PID, termination result and exact matching log lines. If no matching cause evidence is available, record the gap instead of labeling the result OOM.

Step 5. Stop the allocator if it remains active

Run in: VM terminal 1

```
sudo systemctl stop ce-lab02.service 2>/dev/null || true
systemctl is-active ce-lab02.service; echo "unit status=$? (nonzero means not active)"
```

Record: Note the final unit state alongside your saved logs. Stopping a leftover allocator restores the initial condition but does not replace the termination diagnosis.

Recovery check: The allocator is stopped and its termination evidence has been recorded.

Write your answer

Use the observations recorded beside each step.

Case	Termination	Matching cause evidence
Known SIGKILL		
Memory-limited allocator		

1. What exit status did the deliberate SIGKILL produce, and how do you know its cause?
2. What terminated the memory-limited allocator? Cite its PID, experiment time, unit result and matching kernel or systemd-oomd evidence.
3. Why can neither exit status 137 nor the systemd-run exit status diagnose an OOM kill by itself?

Apply the same reasoning

Does free memory elsewhere in the VM rule out a cgroup OOM kill?

Save results before removing this lab's resources and files: `./lab.sh 02 reset`

Lab 03 - Restart policies have limits

Why does one crash recover while a burst of crashes can leave the same service failed?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 03 setup` on the host. No other lab is required.

Theory you need

A supervisor separates a service's desired state from its current process. `Restart=always` asks `systemd` to start a replacement after a crash. It is still subject to start-rate limiting; the request for a restart and permission to start are separate decisions.

This fixture explicitly sets `StartLimitIntervalSec=20` and `StartLimitBurst=4`. Starts consume the allowance, including initial or manual starts; successful uptime does not instantly erase it. A single kill after `reset-failed` can recover, whereas enough closely spaced starts exhaust the allowance and stop automatic recovery.

NGINX sends requests to A or B and can try another usable backend on a connection failure. If A gives up restarting, B may still answer. Measure A's state and client HTTP results independently: successful HTTP through the proxy cannot identify which backend served it.

`reset-failed` clears the failed state and start counter; start requests recovery. Disabling the limit would permit more restart attempts, but would not fix the cause of the crashes. A passing single-crash experiment therefore says little about a repeated-crash condition.

Source: docs/chaos-theory.md: §§2.4-2.6 and Experiment Cards 2.1-2.2.

Experiment

A and B serve through NGINX. The core experiment targets only A: `Restart=always`, at most four starts per 20 seconds. B stays available during this comparison.

Before running: Predict A's state after one kill and after six rapid kill attempts, with `Restart=always` unchanged.

Measurement key

Signal	Meaning
ActiveState / Result	ActiveState shows whether A is running or failed. Result and the journal explain a refused restart.
NRestarts	Counts automatic restarts; it is not the complete count of all starts used by the rate limiter.
Start request repeated too quickly	Evidence that the start-rate limit was reached. A later kill saying the unit is inactive is a consequence, not another successful injection.
HTTP response	A successful response tests the proxy path, not the health of both backends.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 03 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Check baseline and clear the previous start counter

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a
sudo systemctl start ce-lab03-a ce-lab03-b
systemctl is-active ce-lab03-a ce-lab03-b
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts -p Restart -p StartLimitBurst -p
StartLimitIntervalUSec
curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
```

Record: Fill the Baseline row with A's PID, state, result, restart count and proxy HTTP status. Note the four-start, 20-second limit before comparing crash outcomes.

Step 2. Kill A once and inspect its recovery

Run in: VM terminal 1

```
sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a
sleep 1
curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
```

Record: Fill the One crash row and compare MainPID and NRestarts with baseline. Record the HTTP result separately from A's recovery evidence.

Step 3. Repeat crashes rapidly and inspect the refused restart

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a
burst_since=$(date --iso-8601=seconds)
for i in $(seq 1 6); do
  echo "crash attempt=$i"
  sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a || true
  sleep 1
done
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
sudo journalctl -u ce-lab03-a --since "$burst_since" --no-pager
curl -sS --max-time 3 -o /dev/null -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
```

Record: Fill the Crash burst row with A's final state, result, restart count, relevant journal lines and proxy HTTP status. Note whether a start-limit message appeared and which kill attempts succeeded.

Step 4. Recover and prove the proxy works again

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a ce-lab03-b
sudo systemctl start ce-lab03-a ce-lab03-b
systemctl is-active ce-lab03-a ce-lab03-b
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
# The backends need a moment to bind, and NGINX retries a failed backend after fail_timeout=1s.
for i in $(seq 1 10); do curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/ && break;
  sleep 0.5; done
```

Record: Fill the Recovered row with A's final state, result and PID, B's state, and the proxy response. Name reset-failed and start when explaining how the failed service recovered.

Recovery check: Both backends are active and the proxy responds successfully.

Write your answer

Use the observations recorded beside each step.

Phase	A PID / state / result / restarts	HTTP result	Journal evidence
Baseline			
One crash			
Crash burst			

Phase	A PID / state / result / restarts	HTTP result	Journal evidence
Recovered			

1. How did A's PID, state and restart count change after one crash?
2. What happened to A during the rapid crash burst? Cite the unit result and the matching start-limit journal message.
3. What did the proxy return in each phase, and why does that not establish A's health?
4. Which recovery commands restored A, and what proves both backends and the proxy recovered?

Apply the same reasoning

What would disabling the start limit prove, and what defect would remain?

Save results before removing this lab's resources and files: `./lab.sh 03 reset`

Lab 04 - CPU contention and a quota

How does capping a CPU competitor change the time taken by the same job?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 04 setup` on the host. No other lab is required.

Theory you need

A CPU-bound job needs CPU execution time. Its wall time also includes waiting to be scheduled. Keeping the calculation fixed while adding another runnable job on the same CPU tests contention without changing the application's work.

USE means utilization, saturation and errors, checked per resource. CPU utilization says how busy the CPU is; saturation is runnable work waiting for CPU. High utilization alone does not prove a problem. Connect the resource measurements to a change in this job's elapsed time.

CPU placement and CPU bandwidth are different controls. `taskset` pins both jobs to the same allowed CPU. `CPUQuota=20%` caps the neighbour at one fifth of one CPU on average. It neither reserves a core nor promises the remaining time exclusively to the measured job.

In `cgroup v2`, `cpu.max` contains quota and period in microseconds. Divide quota by period to obtain the CPU budget, for example $20000 / 100000 = 0.2$. When the group uses that budget it waits until replenishment; rising throttling counters show enforcement. Compare repeated baseline timings, loaded timings, capped timings and recovery using the same calculation.

Source: docs/chaos-theory.md: §§3.2, 3.3.5 and the CPU-shares correction; §5.5.2.

Experiment

Select one allowed CPU automatically and pin both jobs to it. The neighbour runs in `ce-lab04` for at most 45 seconds. Each measurement uses the same six-iteration job.

Before running: Rank the job time with no competitor, an unrestricted competitor and a competitor capped at 20%. Explain your prediction.

Measurement key

Signal	Meaning
work.py output	Column 1 is iteration number; column 2 is elapsed seconds. Mean is the sum of column 2 divided by the number of rows; range is its minimum to maximum. Compare all samples in each phase, not a single fast sample.
cpu.max / cpu.stat	cpu.max is configuration. nr_throttled counts throttled periods; throttled_usec is accumulated throttling time. Configuration alone does not show that a limit was reached.
CPU PSI / vmstat r	PSI reports time with runnable tasks waiting; r counts runnable tasks. Both are VM-wide supporting signals, not attribution to this one neighbour.
Neighbour active	If it ended before the job finished, the sample mixes loaded and recovered states and must be repeated.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 04 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Choose an allowed CPU

Run in: VM terminal 1

```
cd ~/labs/lab04
CPU=$(python3 -c 'import os; print(min(os.sched_getaffinity(0)))')
echo "Both workloads will use CPU $CPU"
systemctl is-active ce-lab04.service; echo "unit status=$? (nonzero means not active)"
```

Record: Note the selected CPU and confirm no `ce-lab04` neighbour is active before baseline. Keep this VM shell open so CPU stays defined for all later commands.

Step 2. Record the baseline three times

Run in: VM terminal 1

```
cd ~/labs/lab04
for run in 1 2 3; do taskset -c "$CPU" python3 work.py | tee "baseline-$run.txt"; done
awk 'NR==1 {min=max=$2} {sum+=$2; if ($2<min) min=$2; if ($2>max) max=$2} END {printf "baseline mean=%.4fs\n", sum/NR, min, max}' baseline-1.txt baseline-2.txt baseline-3.txt
```

Record: Fill the Baseline row with the combined mean and range. Keep all three baseline files to distinguish a change under contention from ordinary variation.

Step 3. Add one bounded CPU neighbour on the same CPU

Run in: VM terminal 1

```
cd ~/labs/lab04
sudo systemd-run --unit=ce-lab04 --collect -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
systemctl is-active ce-lab04
taskset -c "$CPU" python3 work.py | tee loaded.txt
systemctl is-active ce-lab04 || echo 'Inconclusive: the neighbour ended before the measurement finished.'
cat /proc/pressure/cpu
vmstat 1 5
sudo systemctl stop ce-lab04
```

Record: Save `loaded.txt` and the neighbour's before/after active states. Record CPU PSI and `vmstat r` as supporting observations; mark the phase inconclusive if the neighbour was not active throughout.

Step 4. Cap the neighbour at 20 percent and read throttling counters

Run in: VM terminal 1

```
cd ~/labs/lab04
for i in $(seq 1 10); do
  systemctl list-units --all --plain --no-legend ce-lab04.service | grep -q . || break
  sleep 1
done
sudo systemd-run --unit=ce-lab04 --collect -p CPUQuota=20% -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
path=$(systemctl show ce-lab04 -p ControlGroup --value)
if [ -n "$path" ] && systemctl is-active --quiet ce-lab04; then
  echo 'Neighbour active before measurement'
  sudo cat "/sys/fs/cgroup$path/cpu.max"
  sudo cat "/sys/fs/cgroup$path/cpu.stat" | tee quota-before.txt
  taskset -c "$CPU" python3 work.py | tee quota.txt
  if systemctl is-active --quiet ce-lab04; then
    echo 'Neighbour still active after measurement'
    sudo cat "/sys/fs/cgroup$path/cpu.stat" | tee quota-after.txt
    awk 'NR==FNR {before[$1]=$2; next} $1 ~ /^(nr_throttled|throttled_usec)$/ {print $1 " increase="
      $2-before[$1]}' quota-before.txt quota-after.txt
  else
    echo 'Inconclusive: the neighbour ended before the measurement finished.'
  fi
else
  echo 'STOP: no active neighbour cgroup; this phase was not measured.'
fi
sudo systemctl stop ce-lab04
```

Record: Save quota.txt, the cpu.max values, both cpu.stat snapshots and their printed counter increases. Record whether the neighbour stayed active; the configuration alone is not evidence of enforcement.

Step 5. Recover and repeat the baseline

Run in: VM terminal 1

```
cd ~/labs/lab04
sudo systemctl stop ce-lab04 2>/dev/null || true
for i in $(seq 1 10); do
  systemctl list-units --all --plain --no-legend ce-lab04.service | grep -q . || break
  sleep 1
done
systemctl is-active ce-lab04; echo "unit status=$? (nonzero means not active)"
taskset -c "$CPU" python3 work.py | tee recovered.txt
awk 'NR==1 {min=max=$2} {sum+=$2; if ($2<min) min=$2; if ($2>max) max=$2} END {printf "baseline mean=%.4fs
  range=%.4f-%.4fs\n", sum/NR, min, max}' baseline-1.txt baseline-2.txt baseline-3.txt
for f in loaded.txt quota.txt recovered.txt; do
  awk -v f="$f" 'NR==1 {min=max=$2} {sum+=$2; if ($2<min) min=$2; if ($2>max) max=$2} END {printf "%s
    mean=%.4fs range=%.4f-%.4fs\n", f, sum/NR, min, max}' "$f"
done
```

Record: Fill the remaining mean/range cells using the printed summary and record the recovered neighbour state. Use this comparison and the counter differences to answer the quota question.

Recovery check: ce-lab04 is stopped and the final job timing is recorded.

Write your answer

Use the observations recorded beside each step.

Phase	Mean / range (s)	Neighbour active?	Throttle evidence
Baseline			
Unrestricted neighbour			
20% neighbour			
Recovered			

1. What were the mean and range of iteration times at baseline, with the unrestricted neighbour, with the 20% neighbour and after recovery?
2. Was the neighbour active throughout each loaded measurement? Which measurements, if any, are inconclusive?
3. What quota/period ratio did `cpu.max` show, and how much did the throttling counters increase during the capped measurement?
4. How do the timings, throttling evidence and recovered baseline explain the effect of capping the competitor?

Apply the same reasoning

Would pinning the neighbour to a different otherwise idle CPU test the same contention?

Save results before removing this lab's resources and files: `./lab.sh 04 reset`

Lab 05 - A container is several Linux mechanisms

Why does an isolated process view not tell you how much CPU a sandbox can use?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 05 setup` on the host. No other lab is required.

Theory you need

A Linux container combines mechanisms; it is not a small machine with its own kernel. `chroot` changes where absolute filesystem paths begin. A mount namespace separates mount configuration. A PID namespace gives processes their own PID numbering and visibility.

`unshare --pid --fork` starts a child in a new PID namespace. Its first process is PID 1 there but has an ordinary PID visible from the VM. A fresh `proc` mount is needed for `ps` to display the new view; changing the filesystem root alone does not hide host processes.

Cgroups separately account for and constrain resource consumption. The `systemd` scope places the sandbox in a group with `CPUQuota=20%`, `MemoryMax=128M` and `TasksMax=50`. An isolated `ps` listing says nothing about those budgets.

A CPU quota is time per period: divide the two `cpu.max` numbers. Read `cpu.stat` before and during a busy loop to distinguish a configured cap from actual throttling. The sandbox still shares the VM kernel and, because no network namespace is requested, its network. Each isolation claim needs its own evidence.

Source: docs/chaos-theory.md: §§5.2-5.5.2 and §5.7.1.

Experiment

The prepared BusyBox sandbox uses `chroot`, PID and mount namespaces, and a `systemd` scope with CPU, memory and task limits.

Before running: Predict which observation shows a separate PID view and which shows enforced CPU limiting.

Measurement key

Signal	Meaning
echo \$\$ / ps	PID 1 and the small process list establish the sandbox's PID view. The host can still see these processes.
ls /	Shows the exported BusyBox filesystem used as the sandbox's root.
cpu.max / memory.max / pids.max	cpu.max gives quota and period in microseconds; $20000 / 100000 = 0.2$ of one CPU. memory.max is bytes; 128 MiB = 134217728 bytes. pids.max counts tasks, including threads.
cpu.stat	nr_throttled counts periods with throttling; throttled_usec accumulates throttled microseconds. Compare the later value minus the earlier value. An increase during the loop shows enforcement; an old nonzero total alone does not.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 05 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Confirm the exported BusyBox root filesystem

Run in: VM terminal 1

```
ls ~/labs/lab05/rootfs
systemctl is-active ce-lab05.scope; echo "scope status=$? (nonzero means not active)"
```

Record: Note the prepared root's directory names and the scope's starting state. Keep two VM terminals open for the remaining steps.

Step 2. Start the sandbox under a resource-limited systemd scope (this shell becomes the sandbox)

Run in: VM terminal 1

```
sudo systemd-run --unit=ce-lab05 --scope --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p TasksMax=50 -p CPUQuota=20% \
  unshare --fork --pid --mount --mount-proc="$HOME/labs/lab05/rootfs/proc" \
  chroot "$HOME/labs/lab05/rootfs" /bin/sh
```

Record: Note that terminal 1 is now the sandbox. Leave it open; run host inspection commands only in VM terminal 2 while `ce-lab05.scope` exists.

Step 3. Inspect the view inside the sandbox

Run in: Sandbox shell (terminal 1)

```
echo $$
ps
ls /
```

Record: Fill the Filesystem view and PID view rows with the directory listing, shell PID and visible processes. These observations establish visibility, not resource limits.

Step 4. Read the limits and PID namespaces from the host

Run in: VM terminal 2

```
path=$(systemctl show ce-lab05.scope -p ControlGroup --value)
if [ -n "$path" ] && systemctl is-active --quiet ce-lab05.scope; then
  sudo cat "/sys/fs/cgroup$path/cpu.max" "/sys/fs/cgroup$path/memory.max" "/sys/fs/cgroup$path/pids.max"
  sudo lsns -t pid
else
  echo 'STOP: no active sandbox scope. Restart the sandbox before reading its limits.'
fi
```

Record: Fill the Configured limits row with all three cgroup files and compute quota/period. Record the sandbox namespace's host PID from lsns to compare with the PID reported inside it.

Step 5. Run a finite busy loop inside the sandbox

Run in: Sandbox shell (terminal 1); observe from terminal 2 while it runs

```
timeout 30 sh -c "while ;; do ;; done"
echo "busy-loop exit=$?"
```

Record: While the loop runs, switch immediately to VM terminal 2 for the next step. Note the loop's start and completion; both counter snapshots must be taken while it runs, otherwise repeat both steps.

Step 6. Observe enforcement from the host while the loop runs

Run in: VM terminal 2

```
cd ~/labs/lab05
path=$(systemctl show ce-lab05.scope -p ControlGroup --value)
if [ -n "$path" ] && systemctl is-active --quiet ce-lab05.scope; then
  sudo cat "/sys/fs/cgroup$path/cpu.stat" | tee cpu-before.txt
  sleep 5
  sudo cat "/sys/fs/cgroup$path/cpu.stat" | tee cpu-after.txt
  awk 'NR==FNR {before[$1]=$2; next} $1 ~ /^(nr_throttled|throttled_usec)$/ {print $1 " increase="
    $2-before[$1]}' cpu-before.txt cpu-after.txt
  top -b -n 2 -d 2 | awk '/^top -/ {frame++} frame == 2' | head -15
else
  echo 'STOP: no active sandbox scope. Restart the sandbox before observing enforcement.'
fi
```

Record: Fill the Busy-loop throttling row with the before/after values and printed increases for both counters. Use these deltas to distinguish enforced CPU limiting from the isolated PID view.

Step 7. After the loop finishes, exit the sandbox

Run in: Sandbox shell (terminal 1)

```
exit
```

Record: Note that you exited the sandbox after recording the measurements. Confirm resource cleanup from terminal 2 in the next step.

Step 8. Confirm the scope stopped

Run in: VM terminal 2

```
systemctl is-active ce-lab05.scope; echo "status=$? (nonzero means not active)"
```

Record: Record the final scope state to confirm the experimental process and its resource-limited scope are no longer running.

Recovery check: The sandbox is exited and its scope is no longer active.

Only if normal recovery fails

Step 1. Fallback if the sandbox shell is unresponsive

Run in: VM terminal 2

```
sudo systemctl stop ce-lab05.scope
sudo systemctl reset-failed ce-lab05.scope 2>/dev/null || true
```

Write your answer

Use the observations recorded beside each step.

Observation	Value / change	Responsible mechanism
Filesystem view		
PID view		
Configured limits		

Observation	Value / change	Responsible mechanism
Busy-loop throttling		

1. What filesystem and PID views did the sandbox show, and which mechanisms create those views?
2. What CPU, memory and task limits did the host read from the sandbox's cgroup?
3. How much did `nr_throttled` and `throttled_usec` increase while the busy loop ran? What does that prove beyond the configured quota?
4. Why does an isolated process listing alone tell you nothing about the sandbox's CPU budget?

Apply the same reasoning

Would removing CPUQuota make host processes appear in the sandbox's `ps` output?

Save results before removing this lab's resources and files: `./lab.sh 05 reset`

Lab 06 - Separate containers can share a full filesystem

Why can one container prevent another from writing to a shared filesystem?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 06 setup` on the host. No other lab is required.

Theory you need

Containers can see different root filesystems yet share the same storage capacity. A bind mount exposes a host directory inside a container; mounting that same directory into two containers does not make two copies of its free space.

This experiment uses a 32 MiB `tmpfs`, a small memory-backed filesystem, as the shared capacity pool. It demonstrates the shared-capacity mechanism from the book without filling the VM's root disk. It does not measure disk throughput or reproduce Docker image-layer storage.

A write needs both permission and available capacity. `dd` writes a known number of bytes; when the filesystem cannot hold more, it can write only part of the data and then return `ENOSPC` ("No space left on device"). A fresh container still sees the same full mount.

Use the identical one-MiB probe before filling, while full, and after deleting the filling file. A failure only in the full phase, supported by `df`, connects the symptom to shared capacity. CPU and PID cgroup limits would not create more capacity on this mount.

Source: `docs/chaos-theory.md`: §5.4 and Experiment Card 5.1. The `tmpfs` fixture adapts the shared-storage example.

Experiment

Two disposable containers mount the same 32 MiB `tmpfs`. Only this small lab mount is filled.

Before running: Will a fresh container be able to write after another fills their shared mount?

Measurement key

Signal	Meaning
dd bs=1M count=1	Attempts to write one MiB. Record the write's error directly. In the filling command, df runs after dd, so the container can exit zero even though dd failed.
df -h /data	Shows capacity of the shared mount. Avail approaching zero supports the ENOSPC explanation.
--rm	Removes the container after exit. It does not delete data written into the host bind mount.
mountpoint	A false result after recovery confirms the temporary filesystem was unmounted.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 06 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Mount and identify the bounded shared filesystem

Run in: VM terminal 1

```
mkdir -p ~/labs/lab06/shared
if mountpoint -q "$HOME/labs/lab06/shared"; then
  echo 'STOP: a previous mount remains. Complete recovery or reset before restarting.'
else
  sudo mount -t tmpfs -o size=32m tmpfs "$HOME/labs/lab06/shared"
fi
findmnt --mountpoint "$HOME/labs/lab06/shared" -o TARGET,FSTYPE,SIZE,AVAIL
```

Record: Save the mountpoint, filesystem type and capacity. These establish which bounded resource both containers will share.

Step 2. Baseline - write one MiB before filling

Run in: VM terminal 1

```
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/probe bs=1M count=1'
echo "baseline probe exit=$?"
df -h ~/labs/lab06/shared
sudo rm -f ~/labs/lab06/shared/probe
```

Record: Fill the Empty row with the probe's exit status and available space after the write. The final removal keeps this probe from consuming capacity during the next phase.

Step 3. Fill only the 32 MiB tmpfs from one container

Run in: VM terminal 1

```
if [ "$(findmnt --mountpoint "$HOME/labs/lab06/shared" --noheadings --output FSTYPE)" = tmpfs ]; then
  docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/full bs=1M count=40; echo "filling dd exit=$?"; df -h /data'
else
  echo 'STOP: the dedicated tmpfs is not mounted. Do not run the full-filesystem probe.'
fi
```

Record: Save the filling write's exact error, its own exit status and df capacity. The writer container is removed by `--rm`, but its filling file remains in the bind mount.

Step 4. Repeat the identical probe from a fresh container while full

Run in: VM terminal 1

```
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
echo "full probe exit=$?"
df -h ~/labs/lab06/shared
```

Record: Fill the Full row with the probe status, exact error and available capacity. Compare with the successful baseline despite using the same image, mount and write command.

Step 5. Free capacity and repeat the identical write

Run in: VM terminal 1

```
sudo rm -f ~/labs/lab06/shared/full ~/labs/lab06/shared/probe
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
echo "recovered probe exit=$?"
df -h ~/labs/lab06/shared
```

Record: Fill the Space freed row with the probe status and available space. Compare the recovery action and result with the previous fresh-container attempt.

Step 6. Remove the probe and unmount the temporary filesystem

Run in: VM terminal 1

```
sudo rm -f ~/labs/lab06/shared/probe
sudo umount ~/labs/lab06/shared
mountpoint ~/labs/lab06/shared; echo "status=$? (nonzero means unmounted)"
docker ps -a --filter 'name=~ce-lab06-'
```

Record: Save the mountpoint result and container listing as cleanup evidence after recording the three write outcomes.

Recovery check: No ce-lab06 test container remains and the shared tmpfs is unmounted.

Write your answer

Use the observations recorded beside each step.

Phase	Probe result	Available space / error
Empty		
Full		
Space freed		

1. What did the identical one-MiB write return on the empty filesystem, while full, and after space was freed? Give the write status and available capacity for each phase.
2. What error and capacity measurements connect the failed write to shared-space exhaustion?
3. Why did a fresh container still fail, and which action restored its ability to write?

Apply the same reasoning

Why does deleting the container that filled the mount not recover the space?

Save results before removing this lab's resources and files: `./lab.sh 06 reset`

Lab 07 - Network delay accumulates across sequential calls

How does the same network delay affect one dependency exchange versus four sequential exchanges?

Before you run: Provision the shared VM once with ./lab.sh provision. Run ./lab.sh 07 setup on the host. No other lab is required.

Theory you need

A network namespace holds interfaces, routes and packet queueing configuration. nsenter --net starts a host command in a target process's network namespace. The command uses the host's installed tc tool while changing the container's eth0. No extra tool needs to be installed in the target image.

tc configures a queueing discipline (qdisc), the rule for scheduling outgoing packets on an interface. netem delay 25ms delays each outgoing packet by approximately 25 ms. Here only the dependency's egress is changed; the VM interface and the opposite direction are not independently delayed.

The prepared client sends one byte, waits for its reply, and repeats on the same TCP connection. If a request contains N sequential exchanges, added delay is approximately $N \times d$, with packet and scheduling variation. Timing starts after connecting, so connection establishment is excluded. Compare $N=1$ and $N=4$ after subtracting each case's own baseline. Sequential waits add; independent parallel exchanges can overlap, so their completion time follows the slowest required exchange.

The tc command exits after installing the rule. The rule remains in the network namespace until removed or the interface is destroyed. Inspect qdisc state and counters, measure a near-zero control and the larger delay, then delete the rule and repeat the original measurement.

Source: docs/chaos-theory.md: §§5.8-5.8.1 and Experiment Card 5.5.

Experiment

A Python echo server runs in ce-lab07 on port 8070. The host client measures one or four exchanges per sample. Change only the container's eth0 egress: no delay -> 1 ms control -> 25 ms -> no delay.

Before running: Estimate the added time for one and four sequential exchanges at 25 ms per outgoing reply. Will exiting tc remove the delay?

Measurement key

Signal	Meaning
exchanges / mean_ms	Each sample is the elapsed time for N sequential echo exchanges after connecting. Five samples give a mean and range.
tc -s qdisc show	netem with the requested delay confirms configuration; Sent counters increasing after probes confirm traffic traversed it.
1 ms control	Checks the measurement path with a small injected dose. It includes that dose and timing noise, so it is not a pure measurement of tool overhead.
noqueue after deletion	The fault has been removed. The repeated client measurement checks service recovery separately.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 07 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh` ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Measure the unchanged dependency

Run in: VM terminal 1

```
cd ~/labs/lab07
net() { sudo nsenter --target "$(docker inspect -f '{{.State.Pid}}' ce-lab07)" --net -- tc "$@"; }
net qdisc show dev eth0
python3 probe.py 1 | tee baseline-1.txt
python3 probe.py 4 | tee baseline-4.txt
```

Record: Fill the None row with both means and ranges plus the qdisc state. Keep this VM shell open so the net function remains defined for later steps.

Step 2. Measure a 1 ms control

Run in: VM terminal 1

```
net qdisc add dev eth0 root netem delay 1ms
net -s qdisc show dev eth0 | tee control-before.txt
python3 probe.py 1 | tee control-1.txt
python3 probe.py 4 | tee control-4.txt
net -s qdisc show dev eth0 | tee control-after.txt
```

Record: Fill the 1 ms control row with both means and ranges. Compare Sent packets/bytes before and after the probes and record whether the rule is present after its installer exits.

Step 3. Change only the dose to 25 ms

Run in: VM terminal 1

```
net qdisc change dev eth0 root netem delay 25ms
net -s qdisc show dev eth0 | tee delayed-before.txt
python3 probe.py 1 | tee delayed-1.txt
python3 probe.py 4 | tee delayed-4.txt
net -s qdisc show dev eth0 | tee delayed-after.txt
```

Record: Fill the 25 ms row with both means and ranges, the configured delay and the before/after Sent counters. The final comparison step calculates the added time for each exchange count.

Step 4. Remove the rule and repeat both probes

Run in: VM terminal 1

```
net qdisc del dev eth0 root
net qdisc show dev eth0
python3 probe.py 1 | tee recovered-1.txt
python3 probe.py 4 | tee recovered-4.txt
```

Record: Fill the Removed row with both means and ranges and the final qdisc state. Fault removal and successful echo timing are separate pieces of recovery evidence.

Step 5. Calculate added time from each case's own baseline

Run in: VM terminal 1

```
cd ~/labs/lab07
python3 - <<'PY'
import pathlib, re
def mean(phase, count):
    text = pathlib.Path(f'{phase}-{count}.txt').read_text()
    match = re.search(r'mean_ms=([0-9.]+)', text)
    if not match:
        raise SystemExit(f'Missing successful probe: {phase}-{count}.txt')
    return float(match[1])
for count in (1, 4):
    baseline = mean('baseline', count)
    for phase, dose in (('control', 1), ('delayed', 25), ('recovered', 0)):
        added = mean(phase, count) - baseline
        print(f'{phase}: exchanges={count} added_ms={added:.2f} expected_approximately_ms={count * dose}')
PY
```

Record: Use the printed differences to answer the sequential-delay question. Report actual deltas and compare them with $N \times \text{dose}$, supported by the qdisc counters and recovery measurements.

Recovery check: The container has no netem qdisc and both client measurements return toward their own baseline.

Only if normal recovery fails

Step 1. Remove only this container’s injected queue

```
Run in: VM terminal 2
PID=$(docker inspect -f '{{.State.Pid}}' ce-lab07)
if [ "$PID" -gt 1 ]; then
  sudo nsenter --target "$PID" --net -- tc qdisc del dev eth0 root
fi
```

Write your answer

Use the observations recorded beside each step.

Delay	1 exchange mean / range (ms)	4 exchanges mean / range (ms)	qdisc / traffic evidence
None			
1 ms control			
25 ms			
Removed			

- 1. What were the mean and range for one and four exchanges at baseline, with the 1 ms control, with 25 ms delay, and after removal?
- 2. After subtracting each exchange count's own baseline, how do the 25 ms results compare with $N \times 25$ ms?
- 3. Which qdisc configuration and counter changes prove the intended delay was applied to the measured traffic?
- 4. Did exiting tc remove the delay? Which command removed it, and what measurements demonstrate recovery?

Apply the same reasoning

If four exchanges could run independently in parallel, would their delays necessarily add to four times the dose?

Save results before removing this lab's resources and files: ./lab.sh 07 reset

Lab 08 - A syscall error tests application handling

Why can one request succeed even though its close error stops the server?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 08 setup` on the host. No other lab is required.

Theory you need

A syscall crosses the application/kernel boundary. `write` sends bytes through a file descriptor; `close` releases a descriptor. The application receives a result or error and decides whether to retry, continue or exit. A syscall error and a process exit status are different observations.

`strace -p` attaches to one running process. `-e trace=close` selects the displayed calls; `-e inject=close:error=EIO` skips the selected real calls and makes them report an I/O error. The `INJECTED` marker distinguishes this artificial error from a naturally occurring one. Attaching after startup avoids injecting into unrelated initialization calls.

The server's request path writes the response before closing the connection. Failure on a later close can therefore occur after the client received bytes. Test the triggering request, the process state, and the next request separately; a successful first response cannot establish that the service continues.

Observe a normal request with tracing before injecting. Tracing adds overhead, so this lab tests error handling rather than throughput. The normal trace may include an ignored `fsync` error: seeing an error does not prove that it caused the exit. Follow the actual sequence from injected call to server log to later availability.

On Linux, a real close can release the descriptor even when it reports an error. Skipping close with this injection tests the application's error path, not every real close side effect. Do not infer a descriptor leak from the error code alone or blindly retry close on a descriptor number that may have been reused.

Source: docs/chaos-theory.md: §§6.2-6.4 and Experiment Cards 6.1-6.2.

Experiment

Target only the PID saved in `server.pid`. Use two VM terminals; Ctrl-C in the tracing terminal detaches `strace`.

Before running: Predict the triggering response, process state and next request after close returns EIO.

Measurement key

Signal	Meaning
<code>strace return / INJECTED</code>	The syscall returns -1 with EIO. The marker confirms the injected path was reached.
<code>HTTP code / size_download</code>	Code is the response status and <code>size_download</code> is body bytes received. 000 means no HTTP status was obtained, not a server status code.
<code>wait "\$server" / log</code>	In the shell that started the child, <code>wait</code> reports its exit status. Correlate that with the "error closing socket" message.
Next request	Tests continued service after the triggering request; this is independent of whether the earlier response completed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 08 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh` ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Start the server and save its PID

Run in: VM terminal 1

```
cd ~/labs/lab08
./legacy_server > ~/labs/lab08/server.log 2>&1 & server=$!
echo "$server" > server.pid
sleep 1
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code} bytes=%{size_download}\n' http://127.0.0.1:8080/
echo "server PID=$server"
```

Record: Save the server PID and starting HTTP status/bytes. Keep terminal 1 open; it owns the child used by wait.

Step 2. Attach the normal tracer and leave it running

Run in: VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)
sudo strace -p "$PID" -e trace=write,close,fsync -o ~/labs/lab08/trace.txt
```

Record: Confirm attachment, leave strace running and continue in terminal 1.

Step 3. Send one normal traced request

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
kill -0 "$server" && echo 'server still running'
```

Record: Fill the normal-request HTTP and process columns. Then press Ctrl-C in terminal 2 to detach strace.

Step 4. Read the normal trace after detaching

Run in: VM terminal 2

```
cat ~/labs/lab08/trace.txt
awk -F '(' '{print $1}' ~/labs/lab08/trace.txt | sort | uniq -c
grep -E '= -1' ~/labs/lab08/trace.txt | head
```

Record: Record the write/fsync/close sequence, counts and error returns.

Step 5. Attach the close-error injector and leave it running

Run in: VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)
sudo strace -p "$PID" -e trace=close -e inject=close:error=EIO -o ~/labs/lab08/injected-trace.txt
```

Record: Confirm attachment before sending the next request in terminal 1.

Step 6. Trigger the fault and collect the process result

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
sleep 1
if kill -0 "$server" 2>/dev/null; then echo 'server still running'; else wait "$server"; echo "server exit=$?";
fi
tail -3 ~/labs/lab08/server.log
cat ~/labs/lab08/injected-trace.txt
```

Record: Fill the injected-request row with HTTP/bytes, the close trace line and whether it includes INJECTED, the server log error and actual exit status.

Step 7. Test the next request independently

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w 'next: HTTP=%{http_code} bytes=%{size_download}\n' http://127.0.0.1:8080/
echo "curl exit=$?"
```

Record: Fill the next-request row with HTTP status, bytes and curl exit.

Step 8. Restart, verify HTTP recovery and stop the server

Run in: VM terminal 1, same shell

```
cd ~/labs/lab08
if kill -0 "$server" 2>/dev/null; then kill "$server"; wait "$server"; fi
./legacy_server > ~/labs/lab08/server-restarted.log 2>&1 & server=$!
echo "$server" > server.pid
sleep 1
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
kill "$server"; wait "$server"
rm -f ~/labs/lab08/server.pid
```

Record: Fill the restarted row with HTTP status/bytes. Preserve the first server log and both traces for your answers.

Recovery check: The server responds after restart, then the test process is stopped.

Write your answer

Use the observations recorded beside each step.

Phase	HTTP code / bytes	Process state / exit	Trace or log evidence
Normal traced request			
Injected request			
Next request			
Restarted			

1. What did the normal trace show, and did its fsync error stop the server?
2. For the injected request, connect the close return value, server log and process exit status.
3. Compare HTTP status and response bytes for the triggering request, next request and restarted server. Why can they differ?

Apply the same reasoning

Does this test show that every real close error leaves a file descriptor open?

Save results before removing this lab's resources and files: `./lab.sh 08 reset`

Lab 09 - A syscall denial need not crash the process

What changes when the same process installs a filter that denies getpid?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 09 setup` on the host. No other lab is required.

Theory you need

`seccomp` is a kernel filter on a process's syscall attempts. A filter can allow a call, return a chosen error, or use other actions. This example allows calls by default and adds one rule: `getpid` returns `EACCES`. It installs the policy inside its own process using `libseccomp`.

The program calls `syscall(SYS_getpid)` directly before and after loading the filter. That makes the kernel-boundary test explicit rather than relying on a library's implementation of `getpid`. Before filtering a successful call returns a positive PID; an error through syscall returns `-1` and sets `errno`.

Returning an error is not the same as killing the process. write and process-exit operations remain allowed, so the program can print the denied result and finish. Its exit status reports whether the demonstration's check passed; it is not the `getpid` return value.

A loaded filter applies to the test process and is inherited by descendants. Freeing the userspace libseccomp context does not remove the kernel policy. The process cannot simply undo the filter; ending it ends this experiment. A fresh process launched by the original shell does not inherit a filter installed only in the child.

Source: docs/chaos-theory.md: §§6.2 and 6.5.2 (syscalls and libseccomp).

Experiment

Setup compiles filter.c. Running ./filter changes only that test process's syscall policy. No container is required.

Before running: Predict getpid's result before and after the filter, and whether the program can still print the error.

Measurement key

Signal	Meaning
getpid result	A positive number is a successful PID result. -1 indicates a syscall error; read errno with it.
errno=13	EACCES: the rule's chosen permission-denied error. Only interpret errno after an error return.
exit	0 means the expected denial was observed; 1-3 mean filter setup failed; 4 means the observation did not match the rule.
Seccomp	In /proc/self/status, 0 means disabled and 2 means filter mode. This reports the reading process's state, not proof of which syscall was denied.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 09 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Run the filter and capture its exit status

Run in: VM terminal 1

```
cd ~/labs/lab09
./filter
filter_exit=$?
echo "filter exit=$filter_exit"
```

Record: Record the before-filter PID, after-filter result and errno, and filter exit as separate values.

Step 2. Start a fresh instance from the original shell

Run in: VM terminal 1, same shell

```
cd ~/labs/lab09
./filter
echo "fresh instance exit=$?"
grep '^Seccomp' /proc/self/status
```

Record: Fill the fresh-process row with this new instance before-filter PID, later result/errno and exit. Record the separate Seccomp reading.

Recovery check: The demonstration has exited; the filter was scoped to that process. Reset removes the compiled fixture.

Write your answer

Use the observations recorded beside each step.

Phase	Syscall result	errno / process exit
Before filter		
After filter		
Fresh process		

1. Compare getpid results before and after installing the filter. Which errno accompanied the denied call?
2. Why can the program still print, and what does its exit status mean?
3. What do a fresh direct getpid call and /proc/self/status show about filter scope?

Apply the same reasoning

Would seccomp_release restore getpid inside the filtered process?

Save results before removing this lab's resources and files: ./lab.sh 09 reset

Lab 10 - Ownership, disruption budgets and Service routing

What do ownership, disruption budgets and Service routing each guarantee during Pod replacement?

Before you run: Provision the shared VM once with ./lab.sh provision. Run ./lab.sh 10 setup on the host. No other lab is required.

Theory you need

A Pod groups containers and has its own object identity (UID). A Deployment expresses the desired replica count; its ReplicaSet creates Pods to maintain that count. Deleting one managed Pod leaves the desired count unchanged, so reconciliation creates a different Pod rather than resurrecting the deleted object.

The controller creates the replacement, the scheduler chooses a node, and kubelet starts its containers. These are asynchronous steps. The old Pod may still be terminating while the new one starts; a temporarily larger object count is not itself a failure.

A Service selects Pods by labels and routes through eligible endpoints. Readiness determines whether a Pod is eligible; Running alone is insufficient. Here readiness opens TCP port 8080, whereas the client asks /healthz for an HTTP result. Endpoint changes and existing connections can lag behind Pod changes.

Measure both recovery of replica count and sampled client availability. With 100 probes, sampled availability is successful HTTP responses / 100. Zero failed probes means none of these probes saw failure; requests between samples or other paths remain untested. Goldpinger's local /healthz response is not a measurement of every peer connection.

Follow ownerReferences from Pod to ReplicaSet to Deployment. Labels select resources; ownership identifies the controller responsible for replacement. Kubernetes does not recreate an unmanaged Pod merely because a Service selects it.

A PodDisruptionBudget (PDB) constrains voluntary evictions through the Eviction API. With three healthy selected Pods and minAvailable=3, disruptionsAllowed is zero. An eviction should be refused with a disruption-budget error; Forbidden would instead mean an authorization failure. Direct Pod DELETE

bypasses this check, and a PDB cannot prevent node failures.

After replacement, deliberately change only the Service selector to match no Pods. The Pods can stay Ready while the Service loses eligible endpoints. Restoring the selector repairs discovery without restarting the application. EndpointSlices report addresses and conditions, not a per-request success guarantee.

A failure domain is a set of replicas that one failure can remove together. Replicas on the same node share node failure; separate kind nodes still share the host VM. Replica count alone does not establish resilience to either shared failure.

Source: docs/chaos-theory.md: §§10.4, 10.5.1-10.5.2; Experiment Card 10.1.

Experiment

The chaos cluster has three Goldpinger replicas. The readiness probe checks TCP 8080; HTTP probes sample /healthz through the Service.

Before running: Predict whether the PDB blocks eviction and direct deletion separately. Then predict Pod readiness and HTTP results when only the Service selector is wrong.

Measurement key

Signal	Meaning
Pod name / UID	Compare before and after deletion to identify the replacement. A changed process restart count inside the same Pod would be a different event.
READY / endpointslices	READY reports the configured readiness result. EndpointSlice addresses and readiness show the backend set used for routing.
probes.log	Each row contains a timestamp and HTTP code. Count non-200 rows only after all 100 samples finish.
k helper	Runs kubectl against kind-lab10 in namespace chaos-labs. Node queries are cluster-scoped even though the helper specifies a namespace.
ownerReferences	Read the Pod owner, then that ReplicaSet's owner. A selected Pod and an owned Pod are different relationships.
disruptionsAllowed / eviction error	Wait for the PDB controller to report currentHealthy=3 and disruptionsAllowed=0. Only a budget rejection tests the intended guard.
Service selector / ready endpoints	An extra unmatched selector label makes the selector's AND expression false. Count endpoints with ready=true, including an empty set.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 10 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Checkpoint the prepared chaos cluster

Run in: VM terminal 1

```
source ~/labs/lab10/env.sh
k get nodes
k get pods -l app=goldpinger -o wide
k auth can-i list pods --as=system:serviceaccount:chaos-labs:goldpinger
k auth can-i delete pods --as=system:serviceaccount:chaos-labs:goldpinger
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab10-control-plane)
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: Record Ready count, Pod names/UIDs, Service endpoints, HTTP result and both authorization answers.

Step 2. Trace the Pod, ReplicaSet and Deployment owners

Run in: VM terminal 1, same shell

```
victim=$(k get pods -l app=goldpinger -o jsonpath='{.items[0].metadata.name}')
rs=$(k get pod "$victim" -o jsonpath='{.metadata.ownerReferences[0].name}')
k get pod "$victim" -o jsonpath='{.metadata.uid}{" owner="}{.metadata.ownerReferences}{"\n"}'
k get rs "$rs" -o jsonpath='{.metadata.ownerReferences}{"\n"}'
```

Record: Save victim name/UID, its ReplicaSet owner and that ReplicaSet owner. Keep victim defined for the following steps.

Step 3. Create the disruption budget and attempt eviction

Run in: VM terminal 1, same shell

```
k create pdb lab10-budget --selector=app=goldpinger --min-available=3 --dry-run=client -o yaml | k apply -f -
k wait pdb/lab10-budget --for=jsonpath='{.status.currentHealthy}'=3 --timeout=60s
k wait pdb/lab10-budget --for=jsonpath='{.status.disruptionsAllowed}'=0 --timeout=60s
k get pdb lab10-budget
printf '{"apiVersion":"policy/v1","kind":"Eviction","metadata":{"name":"%s","namespace":"chaos-labs"}}\n'
"$victim" > ~/labs/lab10/eviction.json
k create --raw "/api/v1/namespaces/chaos-labs/pods/$victim/eviction" -f ~/labs/lab10/eviction.json
k get pod "$victim" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: Record currentHealthy, disruptionsAllowed, exact eviction response and the victim UID after the request. Stop the comparison if eviction succeeds or the error is Forbidden.

Step 4. Start 100 probes about 0.2 s apart (start this right before the deletion)

Run in: VM terminal 2

```
source ~/labs/lab10/env.sh
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab10-control-plane)
for i in $(seq 1 100); do
  code=$(curl -s --max-time 1 -o /dev/null -w '%{http_code}' "http://$NODE_IP:30080/healthz")
  printf '%s %s\n' "$(date --iso-8601=ns)" "$code"
  sleep 0.2
done | tee ~/labs/lab10/probes.log
```

Record: Leave this loop running. Immediately run deletion in terminal 1; wait for all 100 rows before calculating results.

Step 5. Directly delete the same Pod despite the PDB; observe replacement for up to 60 seconds

Run in: VM terminal 1, same shell

```
echo "victim=$victim deleted_at=$(date --iso-8601=ns)"
k delete pod "$victim" --wait=false
timeout 60s kubectl --kubeconfig="$HOME/labs/lab10/kubeconfig" --context=kind-lab10 --namespace=chaos-labs get
pods -l app=goldpinger -w
# timeout exit 124 ends observation, not the Kubernetes experiment.
```

Record: Record the deletion timestamp and replacement events. The watch ends automatically after 60 seconds.

Step 6. After all 100 probes finish in terminal 2, collect recovery evidence

Run in: VM terminal 1, same shell

```
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k get pods -l app=goldpinger -o
  custom-columns='NAME:.metadata.name,READY:.status.conditions[?(@.type=="Ready")].status,START:.status.startT
ime,NODE:.spec.nodeName'
k get pod "$victim" 2>&1 | tail -1
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide
samples=$(wc -l < ~/labs/lab10/probes.log)
if [ "$samples" -eq 100 ]; then
  awk '{n++; if ($2 == "200") good++;} END {printf "successes=%d/%d failed=%d availability=%.1f%%\n",
    good,n,n-good,100*good/n}' ~/labs/lab10/probes.log
  grep -v ' 200$' ~/labs/lab10/probes.log | head
else
  echo "Only $samples/100 samples recorded; wait for terminal 2 to finish before counting."
fi
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: After terminal 2 finishes, record Pod UIDs, Ready count, endpoints, failed/100 and availability percentage.

Step 7. Break only Service selection and compare all three signals

Run in: VM terminal 1, same shell

```
k delete pdb lab10-budget --ignore-not-found
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":"lab10-no-match"}}}'
sleep 5
k get pods -l app=goldpinger
k get svc goldpinger -o jsonpath='{.spec.selector}{\n}'
k get endpointslices -l kubernetes.io/service-name=goldpinger -o json | jq '[.items[].endpoints[]? |
  select(.conditions.ready == true)] | length'
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$NODE_IP:30080/healthz"
```

Record: Record selector, Ready Pod count, ready endpoint count and HTTP status for the wrong-selector row.

Step 8. Restore the selector and confirm routing recovery

Run in: VM terminal 1, same shell

```
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
for i in $(seq 1 15); do
  curl -fs --max-time 2 "http://$NODE_IP:30080/healthz" && break
  sleep 1
done
k get pods -l app=goldpinger
k get endpointslices -l kubernetes.io/service-name=goldpinger -o json | jq '[.items[].endpoints[]? |
  select(.conditions.ready == true)] | length'
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
```

Record: Record the Ready Pod count, ready endpoint count and HTTP response for the restored-selector row.

Recovery check: Three replicas and three eligible Service endpoints are restored, HTTP succeeds, and lab10-budget is absent.

Only if normal recovery fails

Step 1. Restore Service selection and remove the experiment PDB

Run in: VM terminal 1, same shell

```
source ~/labs/lab10/env.sh
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
k delete pdb lab10-budget --ignore-not-found
```

Write your answer

Use the observations recorded beside each step.

Phase	Identity / Ready Pods	PDB / ready endpoints	HTTP evidence
Baseline ownership			

Phase	Identity / Ready Pods	PDB / ready endpoints	HTTP evidence
Eviction request			
Direct deletion (100 probes)			
Wrong selector			
Selector restored			

1. Use ownerReferences to explain which controller replaces the deleted Pod and how the UID proves replacement.
2. Compare the eviction request and direct deletion. What does the PDB protect?
3. Calculate failed samples out of 100 and sampled availability. What do these measurements establish?
4. Compare Ready Pods, ready endpoints and HTTP with the wrong and restored Service selectors.

Apply the same reasoning

Would three replicas on the same node establish resilience to losing that node?

Save results before removing this lab's resources and files: `./lab.sh 10 reset`

Lab 11 - Health depends on what you test

How do request budgets and readiness probe choice change the verdict on the same slow dependency?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 11 setup` on the host. No other lab is required.

Theory you need

Goldpinger discovers labelled peer Pods and periodically asks each for an HTTP response. A peer is healthy to a caller only if the operation succeeds within that caller's timeout. A slow but running process can fail that test.

The added Pod contains Goldpinger on port 9090 and Toxiproxy on port 8080. Containers in the Pod share a network namespace, so the proxy forwards to 127.0.0.1:9090. Peers still call port 8080 and traverse the proxy; this leaves the original replicas unchanged. Its app label matches the Service, but it lacks `component=baseline`, so the baseline ReplicaSet does not own or adopt it.

A latency toxic delays traffic in a selected direction. Here it delays upstream application data travelling from the caller through the proxy to Goldpinger. A TCP connection to the proxy can be accepted before that data is forwarded. Kubernetes' TCP readiness probe therefore tests a weaker property than the peer's timed HTTP request.

Compare no delay, 100 ms, and 400 ms against the 300 ms peer budget. Under-budget delay can show up in response time without changing a boolean health result. Over-budget delay can change peer health while TCP readiness remains true. Reports are periodically refreshed, so wait for fresh samples and remove the toxic to test recovery.

A second run changes readiness from TCP to HTTP GET `/healthz` through the same proxy, with `timeoutSeconds=1`, `periodSeconds=2` and `failureThreshold=2`. At 400 ms the peer's 300 ms request can fail while the one-second readiness request succeeds. At 1400 ms both budgets are exceeded; readiness needs consecutive failed samples before changing.

Readiness failure makes this Pod ineligible for ordinary Service traffic; it does not restart a container. A liveness failure could restart it, and a startup probe would defer readiness and liveness until startup succeeds. This fixture intentionally has neither liveness nor startup probes. Record UID and restart counts to distinguish readiness recovery from replacement. Restarting a caller does not remove delay in its dependency. A liveness check tied to that dependency can repeatedly restart otherwise functioning callers and reduce available capacity.

Probe configuration on this standalone Pod cannot be changed in place. Recreate it between the TCP and HTTP comparisons, establish a fresh baseline and record its new UID. Within each comparison, changing only the toxic must not require recreation. Goldpinger discovers labelled Pod IPs directly, so removing an unready Pod from Service routing does not necessarily stop peer pings to it.

Source: docs/chaos-theory.md: §§10.4.4, 10.5.3-10.5.4; Experiment Card 10.2.

Experiment

Use this lab's three Goldpinger replicas. Compare 0, 100 and 400 ms using TCP readiness, then recreate only the extra Pod with one-second HTTP readiness and compare 0, 400 and 1400 ms. The peer budget remains 300 ms throughout.

Before running: Compare the 300 ms peer budget with TCP readiness and one-second HTTP readiness at 400 ms and 1400 ms. Predict endpoint eligibility and restart counts separately.

Measurement key

Signal	Meaning
OK / ms / error / PingTime	The selected peer report says whether its HTTP ping met the budget, its recorded latency and any error. Compare PingTime with the printed observation time for freshness. An absent report is missing evidence, not a failed ping.
Ready	In the TCP comparison it tests only the listener. In the HTTP comparison it tests /healthz through the proxy within one second, with two consecutive failures required to become unready.
Toxiproxy toxic	The API shows the configured direction and dose. Check it before interpreting peer observations.
Repeated samples	Read each original replica directly. That avoids accidentally sampling only one backend through the Service.
HTTP readiness / EndpointSlice ready	Read the slow Pod's own EndpointSlice condition. Other ready replicas can keep the Service available and hide this Pod's failure.
UID / restartCount	A new UID between configurations is deliberate recreation. Within one configuration, unchanged UID and restart counts support readiness-only recovery.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 11 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Connect the proxy and record TCP readiness with no delay

Run in: VM terminal 1

```
source ~/labs/lab11/connect-proxy.sh
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
pings
readiness
```


Record: Fill TCP / 0 ms with each baseline peer report, its PingTime, the slow Pod Ready/endpoint conditions, UID and restart counts.

Step 2. Set 100 ms delay and record the TCP comparison

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics" \
  -d '{"name":"lat","type":"latency","stream":"upstream","toxicity":1,"attributes":{"latency":100,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 8
pings
readiness
```

Record: Fill TCP / 100 ms with the configured toxic, all three peer reports, Ready/endpoint conditions, UID and restart counts.

Step 3. Set 400 ms delay and record the TCP comparison

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics/lat" \
  -d '{"attributes":{"latency":400,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 8
pings
readiness
```

Record: Fill TCP / 400 ms with the same measurements. Compare report PingTime to the printed observation time; repeat pings if reports are stale.

Step 4. Switch only the extra Pod to HTTP readiness and record its baseline

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -X DELETE "$T/proxies/slow/toxics/lat"
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
source ~/labs/lab11/connect-proxy.sh ~/labs/lab11/slow-http.yaml
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
pings
readiness
```

Record: Fill HTTP / 0 ms with the new UID and fresh baseline peer, Ready, endpoint and restart readings. Distinguish this deliberate recreation from later recovery.

Step 5. Set 400 ms delay and record the HTTP comparison

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics" \
  -d '{"name":"lat","type":"latency","stream":"upstream","toxicity":1,"attributes":{"latency":400,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 10
pings
readiness
```

Record: Fill HTTP / 400 ms with the toxic, peer reports, Ready/endpoint conditions, UID and restart counts.

Step 6. Set 1400 ms delay and watch HTTP readiness change

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics/lat" \
  -d '{"attributes":{"latency":1400,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
for i in $(seq 1 6); do date --iso-8601=seconds; readiness; sleep 3; done
pings
```

Record: Fill HTTP / 1400 ms with the toxic and timestamped Ready/endpoint changes, peer reports, UID and restart counts.

Step 7. Remove the toxic and record recovery in the same Pod

Run in: VM terminal 1, same shell

```
curl -fsS --max-time 5 -X DELETE "$T/proxies/slow/toxics/lat"
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
k wait --for=condition=Ready pod/goldpinger-slow --timeout=60s
sleep 8
pings
readiness
```

Record: Fill HTTP / removed with the toxic list, refreshed peer reports, Ready/endpoint conditions, UID and restart counts. Compare its UID with HTTP / 0 ms.

Step 8. Stop the port-forward and remove the extra Pod

Run in: VM terminal 1, same shell

```
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k get pods -l app=goldpinger
```

Record: Record the final Pod list and readiness, the delete result and the rollout result.

Recovery check: The toxic is removed and the original three replicas are Ready.

Write your answer

Use the observations recorded beside each step.

Probe / dose	Peer health / latency	Pod Ready / endpoint ready	UID / restarts
TCP / 0 ms			
TCP / 100 ms			
TCP / 400 ms			
HTTP / 0 ms			
HTTP / 400 ms			
HTTP / 1400 ms			
HTTP / removed			

1. Compare peer health and latency at TCP / 0, 100 and 400 ms. Explain any disagreement with TCP readiness.
2. Compare HTTP / 0, 400 and 1400 ms. How do the 300 ms peer budget and one-second readiness budget explain the results?
3. What happened to the slow Pod endpoint, and can peers still discover its IP directly?
4. Use HTTP / removed, UID and restart counts to distinguish readiness recovery from restarting or replacing the Pod.

Apply the same reasoning

Would adding liveness with the same slow dependency necessarily improve availability?

Save results before removing this lab's resources and files: ./lab.sh 11 reset

Lab 12 - Locate startup failures and measure recovery

Can the startup SLI detect different failure stages, distinguish their causes and measure recovery?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 12 setup` on the host. No other lab is required.

Theory you need

An SLI is a measured service outcome; an SLO is a target for it. This lab measures elapsed time from submitting a fresh Pod and Service to receiving HTTP 200 through that Service. The per-trial target is at most 30 seconds. Pod Running is an earlier, weaker milestone.

Startup includes API submission, scheduling, container startup, readiness and usable routing. The image is cached, so this experiment does not include a cold image download. Repeating a warm-start check cannot establish a cold-start guarantee. With `imagePullPolicy=Always`, the runtime checks the registry to resolve the image to a digest (a content identifier), but can reuse cached layers. Always does not mean every layer is downloaded again; registry access and layer transfer are different startup costs.

Test the checker as well as the workload: a known 45-second startup delay should exceed its 30-second budget, and restoring the normal command should recover. Delete the previous Pod and Service before each trial so an old server cannot answer for the new one.

A completed trial can report `success=0`; that is valid evidence of a missed target. A checker error or stale metrics is not a completed failed trial. The last-run timestamp identifies fresh results. Over many representative trials the fraction meeting the target forms an SLI; these diagnostic trials do not establish a long-term SLO.

The same HTTP deadline miss can have different causes. An unmatched required `nodeSelector` leaves a Pod unscheduled: `PodScheduled=False`, reason `Unschedulable`, and no `nodeName`. A delayed application has a node but cannot yet pass readiness. A wrong Service selector leaves a healthy Ready Pod with zero matching endpoints. Use these intermediate observations to locate the failed stage; `success=0` alone does not identify it.

A container restart reruns the same configured command. It does not change `nodeSelector` or Service labels, and a command containing a 45-second sleep repeats that sleep after restart. Recovery must change the setting responsible for the failed stage.

The checker saves a per-trial JSON snapshot of the Pod, Service, `EndpointSlices` and Pod events before cleanup. Capture occurs after the timed HTTP measurement, so it does not increase the measured startup duration. Each read is a separate observation, not an atomic cluster snapshot; diagnostics errors are recorded explicitly.

Run three healthy trials after removing the faults. The fraction meeting the target is successful completed trials / all completed trials. Keep deliberate fault trials separate from the healthy sample. A small controlled sample tests repeatability, not a production reliability target; a missed trial has no measured eventual time-to-success.

Source: docs/chaos-theory.md: §§11.2, 11.4.1-11.4.3; scheduling in §12.1.1.

Experiment

Setup prepares the checker and manifest. Each invocation creates a fresh workload, waits up to its 30-second budget, records a result and cleans up. The image is preloaded.

Before running: Predict success and the distinguishing Pod/endpoint evidence for delayed startup, impossible placement and wrong Service selection. Predict which faults restarting the container could

repair.

Measurement key

Signal	Meaning
success / elapsed	success=1 means an HTTP 200 was observed within the budget. success=0 means none was observed by the deadline. elapsed is seconds from submission.
checker exit	0 means the checker completed and recorded trials, including misses. Nonzero means execution or cleanup failed; inspect the output before using metrics.
ce_last_run_timestamp_seconds	Unix timestamp of the last completed trial. It must advance; old output cannot represent a new run.
NotFound after cleanup	Both Pod and Service should be absent. An API connection error is not evidence that they were deleted.
PodScheduled / nodeName / Ready	No node and Unschedulable locates placement failure. Assigned but unready locates a later stage. Ready alone does not prove Service routing.
diagnostics/run-N.json	Contains the trial timestamp, measurement and pre-cleanup API evidence. Errors mean the corresponding diagnostic was unavailable; they are not an empty successful query.
Healthy sample SLI	Calculate the fraction from the three run output lines, not start.prom, which contains only the most recent trial.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 12 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Prepare the shell and a reusable diagnostic view

Run in: VM terminal 1

```
cd ~/labs/lab12
source ~/labs/lab12/env.sh
show_trial() {
  jq '{trial,
    node: .pod.spec.nodeName,
    conditions: .pod.status.conditions,
    containers: .pod.status.containerStatuses,
    selector: .service.spec.selector,
    ready_endpoints: (if .endpointslices.diagnostic_error then null else
      [.endpointslices.items[]?.endpoints[]? | select(.conditions.ready == true)] | length end),
    diagnostic_errors: [to_entries[] | select(.value.diagnostic_error?) | {resource: .key, error: .value}],
    events: .events.items}' "$@"
}
```

Record: Use this same terminal for every trial. `show_trial` prints the timestamp, startup outcome and pre-cleanup evidence needed for every table row.

Step 2. Run one healthy baseline

Run in: VM terminal 1, same shell

```
cp startup-good.yaml startup.yaml
bash run.sh | tee runs-normal.txt
checker_exit=${PIPESTATUS[0]}
echo "checker_exit=$checker_exit"
if [ "$checker_exit" -eq 0 ] && [ -f diagnostics/run-1.json ]; then
  cp diagnostics/run-1.json evidence-normal.json
  show_trial evidence-normal.json
  cat metrics/start.prom
else
  echo 'STOP: checker did not complete; do not use old metrics or evidence for this trial.'
fi
k get pod startup-check
k get svc startup-check
```

Record: Fill Normal with success/elapsed, trial timestamp, node/conditions and ready endpoints. Save checker exit, diagnostic errors and both cleanup results. Compare timestamps to the preceding completed trial.

Step 3. Run the 45-second startup-delay fault

Run in: VM terminal 1, same shell

```
cp startup-delayed.yaml startup.yaml
grep -n 'command:' startup.yaml
bash run.sh | tee runs-delayed.txt
checker_exit=${PIPESTATUS[0]}
echo "checker_exit=$checker_exit"
if [ "$checker_exit" -eq 0 ] && [ -f diagnostics/run-1.json ]; then
  cp diagnostics/run-1.json evidence-delayed.json
  show_trial evidence-delayed.json
  cat metrics/start.prom
else
  echo 'STOP: checker did not complete; do not use old metrics or evidence for this trial.'
fi
k get pod startup-check
k get svc startup-check
```

Record: Fill 45 s delay with success/elapsed, trial timestamp, node/conditions and ready endpoints. Save checker exit, diagnostic errors and both cleanup results. Compare timestamps to the preceding completed trial.

Step 4. Run the impossible-placement fault

Run in: VM terminal 1, same shell

```
cp startup-unscheduled.yaml startup.yaml
bash run.sh | tee runs-unscheduled.txt
checker_exit=${PIPESTATUS[0]}
echo "checker_exit=$checker_exit"
if [ "$checker_exit" -eq 0 ] && [ -f diagnostics/run-1.json ]; then
  cp diagnostics/run-1.json evidence-unscheduled.json
  show_trial evidence-unscheduled.json
  cat metrics/start.prom
else
  echo 'STOP: checker did not complete; do not use old metrics or evidence for this trial.'
fi
k get pod startup-check
k get svc startup-check
```

Record: Fill Unmatched nodeSelector with success/elapsed, trial timestamp, node/conditions and ready endpoints. Save checker exit, diagnostic errors and both cleanup results. Compare timestamps to the preceding completed trial.

Step 5. Run the wrong-Service-selector fault

Run in: VM terminal 1, same shell

```
cp startup-selector.yaml startup.yaml
bash run.sh | tee runs-selector.txt
checker_exit=${PIPESTATUS[0]}
echo "checker_exit=$checker_exit"
if [ "$checker_exit" -eq 0 ] && [ -f diagnostics/run-1.json ]; then
    cp diagnostics/run-1.json evidence-selector.json
    show_trial evidence-selector.json
    cat metrics/start.prom
else
    echo 'STOP: checker did not complete; do not use old metrics or evidence for this trial.'
fi
k get pod startup-check
k get svc startup-check
```

Record: Fill Wrong Service selector with success/elapsed, trial timestamp, node/conditions and ready endpoints. Save checker exit, diagnostic errors and both cleanup results. Compare timestamps to the preceding completed trial.

Step 6. Restore the healthy manifest and collect three trials

Run in: VM terminal 1, same shell

```
cp startup-good.yaml startup.yaml
bash run.sh 3 | tee runs-restored.txt
restored_exit=${PIPESTATUS[0]}
echo "checker_exit=$restored_exit"
if [ "$restored_exit" -eq 0 ]; then
    mkdir -p evidence-restored
    for n in 1 2 3; do
        cp "diagnostics/run-$n.json" "evidence-restored/run-$n.json"
        show_trial "evidence-restored/run-$n.json"
    done
    cat metrics/start.prom
else
    echo 'STOP: incomplete restored run; do not calculate a three-trial SLI from old evidence.'
fi
k get pod startup-check
k get svc startup-check
```

Record: Fill restored rows 1-3 separately with success/elapsed, timestamp, node/conditions and endpoints. Record checker exit and final cleanup results.

Step 7. Calculate the restored sample success fraction

Run in: VM terminal 1, same shell

```
if [ "${restored_exit:-1}" -eq 0 ]; then
    awk '/^run=/ {n++; if ($2 == "success=1") good++;}
    END {if (n == 3) printf "healthy sample: %d/%d = %.3f (%.1f%%)\n", good,n,good/n,100*good/n;
        else print "Incomplete sample; inspect checker output"}' runs-restored.txt
else
    echo 'STOP: complete the restored run before calculating the sample SLI.'
fi
```

Record: Report healthy successes / 3 and its fraction/percentage only when all three trials completed. Exclude the deliberate fault trials.

Recovery check: The healthy manifest is restored and no startup-check Pod or Service remains.

Only if normal recovery fails

Step 1. Manual fallback if the loop was killed

Run in: VM terminal 2

```
source ~/labs/lab12/env.sh
k delete -f ~/labs/lab12/startup.yaml --ignore-not-found --grace-period=1
```

Write your answer

Use the observations recorded beside each step.

Trial	success / elapsed	Scheduled / Ready	Eligible endpoints / cause
Normal			
45 s delay			
Unmatched nodeSelector			
Wrong Service selector			
Restored trial 1			
Restored trial 2			
Restored trial 3			

1. Complete all seven rows using fresh success/elapsed, Pod conditions and ready endpoints. Which stage failed in each deliberate fault?
2. How do you distinguish a completed deadline miss, missing diagnostics and a checker execution error?
3. Calculate the restored three-trial success fraction. What does this sample establish?
4. Which commands restore the workload and prove cleanup? Would restarting alone repair the three injected settings?

Apply the same reasoning

Does imagePullPolicy: Always force every image layer to download on each run?

Save results before removing this lab's resources and files: `./lab.sh 12 reset`

Lab 13 - Placement, supervision and replacement

How do cordoning and stopping kubelet differ in scheduling, existing execution and replacement?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 13 setup` on the host. No other lab is required.

Theory you need

Kubelet is the node agent that reports state and asks the container runtime to start or stop containers. The runtime executes them separately. Stopping kubelet removes supervision and reporting without inherently stopping an already-running application.

Missing reports must first be detected. The control plane then marks the node unavailable and applies a NoExecute taint. A Pod's tolerationSeconds says how long it tolerates that taint before eviction begins; it is not measured from the moment kubelet stopped.

This lab explicitly gives node-web 20-second not-ready and unreachable tolerations to shorten the comparison. Total replacement time still includes node detection, eviction processing, scheduling and startup. Do not infer a universal Kubernetes recovery time from this configured example.

A replacement Pod can be created elsewhere while the original process remains on the unsupervised node. The API records desired and reported state; it cannot by itself prove that an unreachable node executed a stop. Compare the original container ID in `crictl` with the new Pod UID, then restore kubelet so the old node can reconcile.

Cordon marks a node unschedulable for ordinary new Pods; it does not evict or stop existing Pods and does not turn off kubelet. A replacement created after you delete a managed Pod must find another eligible node. Uncordon permits future placement again; it does not move existing Pods back.

The node-web workload prefers lab13-worker but does not require it. A preference can be overridden when that node is unavailable; a required hostname constraint could instead leave a replacement Pending. Record placement rather than assuming that replica count guarantees independent failure domains. All kind nodes still share this VM.

Record the Lease renewTime separately from the Node Ready condition. A stale Lease shows missing heartbeats; later conditions and NoExecute taints drive a different stage. Timestamped samples place events between observations; ten-second sampling does not measure exact detection latency.

Source: docs/chaos-theory.md: §§12.1.1-12.1.3 and 12.3.1-12.3.2.

Experiment

First cordon the worker hosting node-web and test replacement. Restore scheduling, then stop kubelet on the replacement's worker. Sample 18 times, sleeping ten seconds between samples; API calls add time. Restore kubelet even if no replacement was observed.

Before running: Predict which operation changes scheduling eligibility, which stops reporting, and whether either immediately kills existing containers. Predict whether uncordon moves the replacement back.

Measurement key

Signal	Meaning
Node Ready / taints	A missing heartbeat can produce Ready=Unknown, displayed as NotReady. Record when the taint appears separately from the stop time.
Pod UID / node	A new UID on another node is a replacement. The old object may remain Terminating until its kubelet returns.
crictl ps	Queries the runtime inside the original kind node directly, independent of kubelet's API reports. Match the recorded original container ID.
20-second toleration	Starts when the matching NoExecute taint applies. It neither stops the old process nor guarantees replacement exactly 20 seconds later.
spec.unschedulable	true blocks ordinary new placement but says nothing about the existing process or node health.
Lease renewTime	Node heartbeats are represented by a Lease in kube-node-lease. Compare renewTime before and during loss of kubelet supervision.
Observed time intervals	Compute stop -> first unavailable sample and first taint -> first replacement sample separately; mark unresolved events as not observed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 13 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Record the original Pod and runtime container

Run in: VM terminal 1

```
source ~/labs/lab13/env.sh
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
date --iso-8601=seconds
echo "original pod=$POD node=$NODE"
k get pod "$POD" -o jsonpath='{.metadata.uid}{"\n"}{.spec.tolerations}{"\n"}'
docker exec "$NODE" crictl ps --name web
```

Record: Fill baseline with timestamp, Pod name/UID, node, the runtime container ID and both 20-second tolerations.

Step 2. Cordon the host and inspect the existing Pod

Run in: VM terminal 1, same shell

```
ORIGINAL_NODE=$NODE
date --iso-8601=seconds
k cordon "$ORIGINAL_NODE"
k get node "$ORIGINAL_NODE" -o jsonpath='{.spec.unschedulable}{"\n"}'
k get pod "$POD" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
docker exec "$ORIGINAL_NODE" systemctl is-active kubelet
docker exec "$ORIGINAL_NODE" crictl ps --name web
```

Record: Fill cordoned with timestamp, unschedulable, Pod UID/node, kubelet state and original runtime ID.

Step 3. Delete the Pod while its node is cordoned

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
k delete pod "$POD" --wait=true --timeout=60s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
NEW_POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NEW_NODE=$(k get pod "$NEW_POD" -o jsonpath='{.spec.nodeName}')
NEW_CID=$(docker exec "$NEW_NODE" crictl ps --name web -q)
echo "after deletion: pod=$NEW_POD node=$NEW_NODE container=$NEW_CID"
```

Record: Fill deleted while cordoned with Pod UID/node, runtime container ID and rollout result. Compare the identities to baseline.

Step 4. Uncordon and save a fresh baseline for the kubelet trial

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
k uncordon "$ORIGINAL_NODE"
sleep 5
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
OLD_CID=$(docker exec "$NODE" crictl ps --name web -q)
echo "kubelet trial: pod=$POD node=$NODE container=$OLD_CID"
OLD_UID=$(k get pod "$POD" -o jsonpath='{.metadata.uid}')
echo "kubelet trial UID=$OLD_UID"
kubectl --kubeconfig="$HOME/labs/lab13/kubeconfig" -n kube-node-lease get lease "$NODE" -o
  jsonpath='{.spec.renewTime}{"\n"}'
```

Record: Fill uncordoned with timestamp, Pod UID/node and runtime container ID; compare to the post-deletion values. Save POD, NODE, OLD_UID, OLD_CID and Lease renewTime for the kubelet trial.

Step 5. Stop kubelet and save command-completion time

Run in: VM terminal 1, same shell

```
docker exec "$NODE" systemctl stop kubelet
STOP_EPOCH=$(date +%s)
printf '%s\n' "$STOP_EPOCH" > ~/labs/lab13/stopped-at.txt
date --iso-8601=seconds
docker exec "$NODE" systemctl is-active kubelet
docker exec "$NODE" crictl ps --name web
```

Record: Fill kubelet stopped with stop-command completion time, kubelet state and whether OLD_CID still runs. Continue immediately to sampling.

Step 6. Sample heartbeats, node state and replacement 18 times

Run in: VM terminal 1, same shell

```
: > ~/labs/lab13/samples.tsv
for i in $(seq 1 18); do
  timestamp=$(date +%s)
  node_json=$(k get node "$NODE" -o json) &&
  pods_json=$(k get pods -l app=node-web -o json) &&
  lease=$(kubectl --kubeconfig="$HOME/labs/lab13/kubeconfig" --context=kind-lab13 -n kube-node-lease
    --request-timeout=5s get lease "$NODE" -o jsonpath='{.spec.renewTime}') || {
    echo "sample $i: API observation failed; no state inferred"
    sleep 10
    continue
  }
  ready=$(printf '%s' "$node_json" | jq -r ' [.status.conditions[]? | select(.type == "Ready") | .status][0] //
    "-" )
  tainted=$(printf '%s' "$node_json" | jq ' [.spec.taints[]? | select(.effect == "NoExecute" and (.key ==
    "node.kubernetes.io/not-ready" or .key == "node.kubernetes.io/unreachable"))] | length > 0')
  replacement=$(printf '%s' "$pods_json" | jq -r --arg uid "$OLD_UID" ' [.items[] | select(.metadata.uid != $uid
    and .metadata.deletionTimestamp == null) | .metadata.uid] | if length == 0 then "-" else join(",") end')
  original_running=unknown
  if runtime_ids=$(docker exec "$NODE" crictl ps --name web -q); then
    original_running=no
    if printf '%s\n' "$runtime_ids" | grep -Fxq "$OLD_CID"; then original_running=yes; fi
  fi
  printf '%s\t%s\t%s\t%s\t%s\t%s\n' "$timestamp" "$ready" "$tainted" "$replacement" "$lease"
  "$original_running" >> ~/labs/lab13/samples.tsv
  date --date="@$timestamp" --iso-8601=seconds
  printf 'Ready=%s NoExecute=%s replacement=%s Lease=%s original-running=%s\n' "$ready" "$tainted"
    "$replacement" "$lease" "$original_running"
  printf '%s' "$pods_json" | jq '.items[] | {name: .metadata.name, uid: .metadata.uid, node: .spec.nodeName,
    deleting: .metadata.deletionTimestamp}'
  sleep 10
done | tee ~/labs/lab13/timeline.txt
```

Record: Fill Lease/taint and replacement rows from timestamped observations. samples.tsv stores epoch, Ready, NoExecute, replacement UID, Lease and original-running. Missing observations stay unknown.

Step 7. Restore kubelet and check convergence

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
docker exec "$NODE" systemctl start kubelet
k wait --for=condition=Ready "node/$NODE" --request-timeout=0 --timeout=180s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get nodes
k get pods -l app=node-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName,DELETING:.metadata.deletionTimestamp'
docker exec "$NODE" crictl ps --name web
k get nodes -o custom-columns=NAME:.metadata.name,UNSCHEDULABLE:.spec.unschedulable
echo "original runtime ID=$OLD_CID"
```

Record: Fill restored with timestamp, node readiness, Pod UID/node, unschedulable flags and whether OLD_CID remains. Recheck runtime state if cleanup is still converging.

Step 8. Calculate the two observed intervals

Run in: VM terminal 1, same shell

```
STOP_EPOCH=$(cat ~/labs/lab13/stopped-at.txt)
awk -F '\t' -v stopped="$STOP_EPOCH" '
    $2 != "True" && $2 != "-" && !unavailable {unavailable=$1}
    $3 == "true" && !tainted {tainted=$1}
    $4 != "-" && !replacement {replacement=$1}
    END {
        if (unavailable) printf "stop-command completion to first unavailable sample: %d s\n", unavailable-stopped;
        else print "node unavailable: not observed";
        if (tainted && replacement && replacement >= tainted) printf "first taint sample to first replacement sample: %d s\n", replacement-tainted;
        else if (tainted && replacement) print "taint-to-replacement interval: inconclusive observation order";
        else print "taint-to-replacement interval: not observed";
    }' ~/labs/lab13/samples.tsv
```

Record: Report stop-command-completion-to-unavailable and taint-to-replacement sample intervals. Reads are separate observations with roughly ten-second sampling plus API delay; preserve not observed or inconclusive results.

Recovery check: Kubelet is running on every worker, four nodes are Ready and uncordoned, and one node-web Pod is Ready. If replacement occurred, confirm the recorded old container is gone.

Only if normal recovery fails

Step 1. Restore kubelet even if the Kubernetes API is unavailable

Run in: VM terminal 2

```
for node in lab13-worker lab13-worker2 lab13-worker3; do
    docker exec "$node" systemctl start kubelet
done
```

```
# After the API responds, restore placement eligibility too.
source ~/labs/lab13/env.sh
k uncordon lab13-worker lab13-worker2 lab13-worker3
```

Write your answer

Use the observations recorded beside each step.

Event	Time	Node / Pod API state	Original container running?
Baseline			
Cordoned (same UID)			
Deleted while cordoned			
Uncordoned (no automatic move)			
Kubelet stopped			
Lease stale / taint observed			
Replacement observed			
Kubelet restored			

1. Compare Pod UID/node and runtime identity before cordon, after cordon, after deletion and after uncordon.
2. Build the kubelet-failure timeline from stop-command completion, Lease, first unavailable/taint observations and replacement UID. Calculate the two observed intervals.
3. Did the original container keep running while replacement occurred? What proves the difference between API state and actual execution?

4. What evidence confirms recovery, and what remains unknown if no replacement was observed?

Apply the same reasoning

Why would deleting the old Pod object forcibly be insufficient to prove there is only one running copy?

Save results before removing this lab's resources and files: `./lab.sh 13 reset`

Lab 14 - Quorum, convergence and application availability

How does quorum loss affect committed configuration, Deployment convergence and existing HTTP traffic?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 14 setup` on the host. No other lab is required.

Theory you need

etcd stores Kubernetes state and uses Raft consensus. A change needs a majority of the configured voting members: $\text{floor}(n/2) + 1$. For three members that is two. Stopping a member does not remove it from membership or lower the required majority. floor rounds down; failure tolerance is n minus the required majority, assuming the survivors can communicate.

A leader coordinates committed changes. Losing one member can cause a transition or election but leaves a possible majority. Losing two prevents new commits. A minority cannot accept independent changes without risking conflicting histories.

The management path is `kubectl` -> API server -> etcd. The existing application path is HTTP client -> NodePort routing -> already-running web Pod. That routing need not perform an etcd write for every request, so these two paths can have different availability during quorum loss.

Test a real ConfigMap change, not only a read that might be cached, and separately probe HTTP. Move a static etcd manifest out of kubelet's watched directory to stop that member. Restore the same manifests through Docker, a path independent of the Kubernetes API. This recovers temporarily stopped members with intact data; it is not a backup-restore experiment.

A successful scale request commits a desired replica count; it does not mean those replicas are already Ready. Compare `spec.replicas` with `status.readyReplicas` and wait for rollout completion. This separates the storage operation from controller, scheduler and kubelet convergence.

A client timeout is an unknown result: the caller lacks an acknowledgement and must not infer that no change committed. After recovery, read the ConfigMap value and Deployment spec before overwriting them. Restore an idempotent target, meaning it is safe to repeat: set replicas to two rather than incrementing on each retry. Verify the observed result. A successful GET during disruption can be cached or use a different path and is weaker evidence than a new committed write.

Record etcd member identities and leader before and after losing one member. This lab stops a fixed member; a leader change is only expected if the stopped member was leader or another election occurred. Do not label every single-member failure a leader-election experiment. Two surviving voters suffice regardless of which member was previously leader.

Source: docs/chaos-theory.md: §§12.1.1, 12.1.4 and 12.3.3-12.3.4.

Experiment

Use Lab 14's own lab14 cluster. Move static etcd manifests to stop members and restore the same manifests to recover them; Docker provides a fallback independent of `kubectl`.

Setup points control-plane kubelets at the HA API load balancer, so restoring a local etcd member is not tied to that member's unavailable API server.

Before running: Calculate quorum for 3, 4 and 5 voting members. Predict ConfigMap writes, scaling convergence and HTTP for one and two stopped members; predict whether a leader change is required for the fixed first target.

Measurement key

Signal	Meaning
endpoint status / health	Status identifies members and leader. Health checks test whether the endpoint can participate successfully; read all three after restoration.
cricctl ps --name etcd	Confirms the selected member actually stopped. Moving the file alone is only the injection request.
ConfigMap patch	Change the state value each phase. A successful committed change exercises the write path; a no-op or cached get is weaker evidence.
HTTP code	200 from the recorded worker NodePort measures the existing data path, independently of kubectl.
spec.replicas / readyReplicas	Desired replica count is committed configuration; Ready count is asynchronous workload convergence. A scale command alone measures only the former.
ConfigMap value / resourceVersion	Read the value after recovery to resolve an uncertain write. resourceVersion is an opaque version identifier, not a number for timing or arithmetic.
Member ID / leader / raft term	Identify whether the chosen stopped member was leader before attributing an election to that fault.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 14 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Calculate quorum and establish the baseline

Run in: VM terminal 1

```
for voters in 3 4 5; do
  quorum=$((voters / 2 + 1))
  echo "voters=$voters quorum=$quorum tolerated_failures=$((voters - quorum))"
done
source ~/labs/lab14/helpers.sh
h get nodes
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
PORT=$(h get svc quorum-web -o jsonpath='{.spec.ports[0].nodePort}')
IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab14-worker)
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$IP:$PORT/"
h patch configmap quorum-probe --type merge -p '{"data":{"state":"baseline"}}'
h get configmap quorum-probe -o jsonpath='{.data.state}{ " resourceVersion="}{.metadata.resourceVersion}{ "\n"}'
ec lab14-control-plane endpoint status --cluster -w table
ec lab14-control-plane member list -w table
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

Record: Record quorum/failure-tolerance calculations and the baseline row: stored value/version, desired/Ready replicas, HTTP status, member identities and current leader.

Step 2. Stop one member and test writes, HTTP and leader state

Run in: VM terminal 1, same shell

```
stop_etcd lab14-control-plane2
echo "stop member exit=$?"
docker exec lab14-control-plane2 crictl ps --name etcd
if ! h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-down"}}'; then
  echo 'First write failed; wait five seconds and retry the same idempotent value once.'
  sleep 5
  h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-down"}}'
fi
h get configmap quorum-probe -o jsonpath='{.data.state}{ " resourceVersion="}{.metadata.resourceVersion}{ "\n"}'
curl -sS --max-time 3 -o /dev/null -w '%{http_code}\n' "http://$IP:$PORT/"

ec lab14-control-plane endpoint status -w table
ec lab14-control-plane3 endpoint status -w table
```

Record: Record stop exit and runtime list before interpreting the fault. Fill the two-voter write/value, HTTP and leader fields; compare the stopped member ID to the baseline leader. If stop member exit is nonzero or the target still runs, use recovery before continuing.

Step 3. Scale with two voters and observe convergence

Run in: VM terminal 1, same shell

```
h scale deployment/quorum-web --replicas=3
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

Record: Complete the two-voter row with the scale response, rollout result and desired/Ready replica counts.

Step 4. Stop the second member and compare writes with HTTP

Run in: VM terminal 1, same shell

```
stop_etcd lab14-control-plane3
echo "stop member exit=$?"
docker exec lab14-control-plane3 crictl ps --name etcd
h patch configmap quorum-probe --type merge -p '{"data":{"state":"two-down"}}'
echo "ConfigMap write exit=$?"
h scale deployment/quorum-web --replicas=1
echo "scale exit=$?"
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$IP:$PORT/"
```

Record: Record stop exit/runtime list, both write exits/errors, any readable desired/Ready state and the independent HTTP status for the one-voter row. If stop member exit is nonzero or the target still runs, use recovery before continuing.

Step 5. Restore the two members and wait for endpoint health

Run in: VM terminal 1, same shell

```
start_etcd lab14-control-plane2
start_etcd lab14-control-plane3
healthy=0
for i in $(seq 1 18); do
  if ec lab14-control-plane endpoint health --cluster; then healthy=1; break; fi
  sleep 5
done
test "$healthy" -eq 1 || echo 'STOP: use fallback; do not claim quorum recovery.'
echo "healthy=$healthy"
```

Record: Record whether all endpoint health checks succeed and the final healthy flag. Preserve all errors if the health loop expires.

Step 6. Read persisted values before restoring configuration

Run in: VM terminal 1, same shell

```
observed=0
for i in $(seq 1 18); do
  if h get configmap quorum-probe -o jsonpath='{.data.state}{\n\n}{.metadata.resourceVersion}{\n\n}' &&
    h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas; then
    observed=1; break
  fi
  sleep 5
done
test "$observed" -eq 1 || echo 'STOP: retain the unknown result; do not overwrite it before readback works.'
ec lab14-control-plane endpoint status --cluster -w table
echo "observed=$observed"
```

Record: Fill restored before overwriting with the stored ConfigMap value/version, desired/Ready replicas and leader status. Record observed=1 only when both reads succeed.

Step 7. Restore the explicit baseline and verify all paths

Run in: VM terminal 1, same shell

```
if [[ ${healthy:-0} -eq 1 && ${observed:-0} -eq 1 ]]; then
  h patch configmap quorum-probe --type merge -p '{"data":{"state":"recovered"}}'
  h scale deployment/quorum-web --replicas=2
  h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
  h get configmap quorum-probe -o jsonpath='{.data.state}{\n\n}'
  h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
  ec lab14-control-plane endpoint health --cluster
  curl -fsS --max-time 3 "http://$IP:$PORT/" >/dev/null && echo 'HTTP recovery ok'
else
  echo 'STOP: complete quorum recovery and readback in the previous step before overwriting state.'
fi
```

Record: Fill the final row with stored value, desired/Ready replicas, all endpoint health results and HTTP response. If the guard stops, complete recovery/readback first.

Recovery check: All three etcd endpoints are healthy, the recovered ConfigMap value is readable, desired and Ready replicas are two, and HTTP succeeds.

Only if normal recovery fails

Step 1. Independent fallback that restores the manifests without kubect

Run in: VM terminal 2

```
for node in lab14-control-plane2 lab14-control-plane3; do
  docker exec "$node" sh -c 'test -f /root/ce-etcd.yaml && mv /root/ce-etcd.yaml
    /etc/kubernetes/manifests/etcd.yaml; ls /etc/kubernetes/manifests'
done
```

Write your answer

Use the observations recorded beside each step.

Phase	Write / stored value	Desired / Ready replicas	HTTP / leader evidence
3 voters / baseline 2 replicas			
2 voters / scale to 3			
1 voter / attempted scale to 1			
Restored before overwriting			
Restored target 2 replicas			

1. Calculate quorum and tolerated failures for 3, 4 and 5 configured voting members.

2. With one member stopped, compare the write/readback, replica convergence, HTTP and member/leader evidence.
3. With two members stopped, compare both write attempts and existing HTTP. What does a client timeout establish?
4. What did the recovered stored values show before you overwrote them, and what proves the final baseline was restored?

Apply the same reasoning

Would four configured members tolerate two failures?

Save results before removing this lab's resources and files: `./lab.sh 14 reset`

Lab 15 - Count how layered retries multiply work

How much extra backend work do layered retries create when the same dependency keeps failing?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 15 setup` on the host. No other lab is required.

Theory you need

An original user request is not the same as an attempt. One retry means up to two attempts. When a client retries a proxy call and the proxy retries each upstream call, the per-layer attempt limits multiply: at most $\text{client attempts} \times \text{proxy attempts} \times \text{backend arrivals per original request}$.

Here HTTP 503 means Service Unavailable. NGINX can make two upstream attempts for that response, and the client makes either one or two proxy attempts. Both upstreams deliberately return 503 throughout the fault phase. A successful attempt stops retries early, so the maximum is reached only when the earlier attempts continue failing.

Measure amplification as $\text{backend arrivals} / \text{original requests}$, and useful work as $\text{successful original requests} / \text{original requests}$. More attempts do not imply more successes. Keep the original request count and failure condition identical when changing the client retry count.

Layered retry amplification is the first step in the book's emergent retry-storm example. This small, sequential experiment isolates multiplication; it does not simulate overloaded worker queues or prove self-sustaining failure. In a real system, retries of slow requests may add work while earlier attempts are still running; bounded attempts, a total deadline and backoff address different parts of that problem.

Source: docs/chaos-theory.md: §1.2.3 (emergent retry amplification), §12.1.4 (ingress retry accumulation). This fixture isolates attempt multiplication using HTTP 503.

Experiment

Each case sends 20 sequential requests in three phases: healthy, both upstreams returning HTTP 503, and recovered. NGINX allows two upstream attempts. Repeat with zero and one client retry. The runner owns its proxy inside `ce-lab15.service`, has a 60-second runtime limit, and removes the fault and processes on exit. Reset stops that unit and its children.

Before running: For 20 requests, calculate the maximum backend arrivals with zero versus one client retry, given two proxy attempts. Predict success counts during a persistent 503 fault.

Measurement key

Signal	Meaning
original / client attempts	Each case has 20 original requests. Count all proxy attempts, including retries, separately.
backend arrivals	Sum the two upstream request logs within the phase. Probe and health requests are excluded.
amplification	backend arrivals / 20. Compare it with the theoretical upper bound; fewer arrivals can indicate early success or a failure before reaching a backend.
successes	Counts original requests ending in HTTP 200, not the number of attempts. Compare outcomes alongside cost.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 15 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Run the three phases with zero client retries

Run in: VM terminal 1

```
cd ~/labs/lab15
sudo systemd-run --unit=ce-lab15 --collect --wait --pipe --uid="$(id -u)" --property=RuntimeMaxSec=60s
  --working-directory="$PWD" /bin/bash "$PWD/run-case.sh" 0 | tee no-client-retry.txt
echo "runner exit=${PIPESTATUS[0]} (0 means completed; nonzero means failure or the 60-second limit)"
```

Record: Copy the three phase summaries from `no-client-retry.txt` into the `no-client-retry` rows. Record the runner exit; stop if it is nonzero.

Step 2. Change only the client retry count from zero to one

Run in: VM terminal 1, same shell

```
cd ~/labs/lab15
sudo systemd-run --unit=ce-lab15 --collect --wait --pipe --uid="$(id -u)" --property=RuntimeMaxSec=60s
  --working-directory="$PWD" /bin/bash "$PWD/run-case.sh" 1 | tee one-client-retry.txt
echo "runner exit=${PIPESTATUS[0]}"
```

Record: Copy the three phase summaries from `one-client-retry.txt` into the `one-client-retry` rows. Record the runner exit and compare fault-phase counts with step 1.

Step 3. Compare the six summaries and confirm cleanup

Run in: VM terminal 1, same shell

```
cd ~/labs/lab15
grep '^phase=' no-client-retry.txt one-client-retry.txt
test ! -e fail && echo 'fault marker removed'
ss -ltn '( sport = :8090 or sport = :9001 or sport = :9002 )'
```

Record: Calculate backend arrivals / 20 for both fault phases. Record success fractions, whether the fail marker is absent, and whether `ss` shows any listening sockets below its header.

Recovery check: Each runner removes the fail marker and stops its own upstreams and proxy. Both recovered phases should return 20/20 successes.

Write your answer

Use the observations recorded beside each step.

Case / phase	Client attempts	Backend arrivals / amplification	Successes / 20
No client retry: baseline			
No client retry: 503 fault			
No client retry: recovered			
One client retry: baseline			
One client retry: 503 fault			
One client retry: recovered			

1. For each of the six case/phase rows, record original requests, client attempts, backend arrivals, amplification and successful requests.
2. Calculate the maximum backend arrivals during persistent HTTP 503 for zero and one client retry. Compare these bounds with your measurements.
3. Explain why the extra attempts may add no successful requests, why recovery reduces attempts, and what the cleanup checks show.

Apply the same reasoning

If both layers permit three retries, what is the maximum amplification?

Save results before removing this lab's resources and files: `./lab.sh 15 reset`

Lab 16 - Design one clear experiment

How would you test one resilience claim so the result has a clear meaning?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 16 setup` on the host. No other lab is required.

Theory you need

An experiment tests a specific service claim under a specific fault. Start with a user-visible measurement: for example, successful HTTP responses divided by total probes, or response time in milliseconds. An SLI is that measured quantity; an SLO is the target you choose for it.

Write the baseline and hypothesis using the same operation, load and measurement window. For example: "With one of three replicas unavailable for 20 seconds, at least 99 of 100 probes return HTTP 200 within one second." This states a condition and a falsifiable outcome; replace the numbers with ones justified by your system.

Specify the exact target and dose, and how to prove the fault reached it. A selector matching nothing gives a misleading pass; one matching too much changes the experiment. A control with no fault and a recovery measurement help distinguish the injected cause from an already-broken baseline.

Define a stop threshold, the command that removes the fault, and a fallback that works when the normal control path fails. Analysis must connect measurements to a mechanism and limit the conclusion to the tested conditions. This lab produces a plan, so leave observations unclaimed until an authorized run supplies them.

Source: docs/chaos-theory.md: §§1.3, 2.5; Appendix C; Master Cheat Sheet §6.

Experiment

Fill the prepared card for one service and one fault. Nothing is deployed or disrupted in this lab.

Before running: State what you expect to measure under one fault and why the service should behave that way.

Measurement key

Signal	Meaning
Baseline / hypothesis	Use a named measurement with units, load and duration in both. "Healthy" without a criterion is not enough.
Fault confirmation	Name the evidence that proves the intended target and dose were affected.
Stop / recovery	Stop threshold tells the operator when to abort. Recovery criteria tell them whether service was restored; these are different checks.
Blank-field check	Finds empty fields only. A reviewer must still judge whether the plan is measurable and executable.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 16 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Read the card and choose one service and one fault

Run in: VM terminal 1

```
cd ~/labs/lab16
cat experiment-card.md
```

Record: Choose the service/environment and one fault. Map the card fields to the four answer prompts before editing.

Step 2. Write the complete plan, including executable commands

Run in: VM terminal 1, same shell

```
cd ~/labs/lab16
${EDITOR:-nano} experiment-card.md
```

Record: Write an answer after each field's colon. Include the exact baseline, injection, fault-confirmation, rollback, fallback and recovery commands; label where each runs and define all variables. Add load, duration, units and pass/stop thresholds. In nano, save with `Ctrl+O`, `Enter`, then exit with `Ctrl+X`.

Step 3. Check missing fields and review the saved plan

Run in: VM terminal 1, same shell

```
cd ~/labs/lab16
grep -nE ':[[:space:]]*$' experiment-card.md
check_status=$?
case "$check_status" in
  0) echo 'Fill the fields listed above.' ;;
  1) echo 'No blank fields found; review the answers for clarity.' ;;
  *) echo 'Could not read the card; fix the file error before continuing.' >&2 ;;
esac

printf '\nSaved plan for review:\n'
cat experiment-card.md
```

Record: Record the blank-field check result. Review the displayed card against all four answer prompts, edit again if needed, and retain the completed plan as your evidence.

Recovery check: The card answers each field clearly; this is a design review, not a live test.

Write your answer

Use the observations recorded beside each step.

Design element	Your choice	How another operator checks it
Measurement / baseline		
Fault / target / duration		
Prediction / threshold		
Stop / rollback / recovery		

1. Name one service and environment. Specify the measurement, units, load, sampling window and exact baseline measurement command.
2. Specify one fault, its exact target and maximum duration, the injection and confirmation commands, and your predicted measurement threshold.
3. Specify an observable stop threshold, the rollback and fallback commands, and the command and criterion that prove recovery.
4. Identify what the plan cannot establish. Review every field for completeness without claiming unmeasured results.

Apply the same reasoning

Can a completed card or a passing no-fault control establish resilience to the proposed disruption?

Save results before removing this lab's resources and files: `./lab.sh 16 reset`

Lab 17 - Signal delivery and the shutdown budget

Why can the same application shut down cleanly in one container and be killed in another?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 17 setup` on the host. No other lab is required.

Theory you need

Docker stop sends the configured stop signal to the container's main process, PID 1. After the stop timeout it forcibly kills a container that has not exited. Here the signal is explicitly SIGTERM; the worker handles it, spends two seconds cleaning up, writes a marker and exits zero.

A shell wrapper can sit between Docker and the application. This fixture deliberately ignores TERM in the wrapper and starts Python as a child. The child's own handler is installed, but Docker's signal does not reach it. Seeing a handler in application code is therefore insufficient; inspect the process relationship.

The shell exec builtin replaces the shell with the application without creating a child. It gives the worker PID 1 in the second configuration. Exec fixes this delivery path; it does not add a handler or make slow cleanup faster. A general-purpose init can reap orphaned children, but cannot repair an application that ignores its own termination signal.

SIGTERM (15) can be handled; SIGKILL (9) cannot be caught for cleanup. A one-second stop budget cannot accommodate this worker's two-second cleanup, even with correct delivery. Compare TERM receipt with completion. Exit 137 corresponds to $128 + 9$, consistent with SIGKILL, but does not identify its cause; check Docker's OOMKilled flag and the injected operation.

Container restart policies concern what happens after exit. They neither forward signals nor extend the shutdown budget. This experiment disables automatic restart and retains stopped containers so logs and exit evidence remain readable.

Source: docs/chaos-theory.md: §§5.13.1-5.13.2.

Experiment

Compare a non-forwarding wrapper with a five-second budget, exec with the same budget, then exec with one second. Only one comparison variable changes at a time.

Before running: Predict the worker PID, TERM receipt, cleanup completion and exit status in all three cases.

Measurement key

Signal	Meaning
ready pid / ppid	The startup log identifies whether the worker is PID 1 or a child. Wait for this log before stopping it.
received / cleanup-complete	Receipt proves delivery to Python. The completion line and file prove this fixture finished cleanup; absence alone needs the exit and timeout evidence.
ExitCode / OOMKilled	Separate a successful handler exit from a forced stop. False OOMKilled plus the controlled stop distinguishes this kill from the memory experiment.
stop --timeout	The grace budget in seconds, not an application request timeout. The Docker command itself can succeed even when the container was killed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 17 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Start the wrapper case and confirm the worker is ready

Run in: VM terminal 1

```
docker create --name ce-lab17-wrapper --stop-signal SIGTERM python:3.12-slim sh -c 'trap "" TERM; python -u /worker.py & wait'
docker cp ~/labs/lab17/worker.py ce-lab17-wrapper:/worker.py
docker start ce-lab17-wrapper
for i in $(seq 1 30); do docker logs ce-lab17-wrapper 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-wrapper
```

Record: Record the ready line, worker PID and parent PID for Wrapper / 5 s. Continue only when ready appears.

Step 2. Stop the wrapper with five seconds and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 5 ce-lab17-wrapper
docker logs ce-lab17-wrapper
docker inspect -f '{{json .State}}' ce-lab17-wrapper
docker cp ce-lab17-wrapper:/cleanup-complete ~/labs/lab17/wrapper-complete.txt
```

Record: Complete Wrapper / 5 s: elapsed stop time, presence/absence of received=15 and cleanup-complete, ExitCode, OOMKilled and marker-copy result. Record any copy error.

Step 3. Start the exec case with the same application

Run in: VM terminal 1, same shell

```
docker create --name ce-lab17-exec --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
docker cp ~/labs/lab17/worker.py ce-lab17-exec:/worker.py
docker start ce-lab17-exec
for i in $(seq 1 30); do docker logs ce-lab17-exec 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-exec
```

Record: Record the ready line and worker PID for Exec / 5 s. Continue only when ready appears; compare the start command with the wrapper case.

Step 4. Stop exec with five seconds and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 5 ce-lab17-exec
docker logs ce-lab17-exec
docker inspect -f '{{json .State}}' ce-lab17-exec
docker cp ce-lab17-exec:/cleanup-complete ~/labs/lab17/exec-complete.txt
cat ~/labs/lab17/exec-complete.txt
```

Record: Complete Exec / 5 s with elapsed stop time, TERM/cleanup logs, marker contents, ExitCode and OOMKilled. Compare with the wrapper using the same budget.

Step 5. Start another exec case for the shorter budget

Run in: VM terminal 1, same shell

```
docker create --name ce-lab17-short --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
docker cp ~/labs/lab17/worker.py ce-lab17-short:/worker.py
docker start ce-lab17-short
for i in $(seq 1 30); do docker logs ce-lab17-short 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-short
```

Record: Record the ready line and worker PID for Exec / 1 s. Confirm the application and exec command match the five-second case.

Step 6. Stop exec with one second and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 1 ce-lab17-short
docker logs ce-lab17-short
docker inspect -f '{{json .State}}' ce-lab17-short
docker cp ce-lab17-short:/cleanup-complete ~/labs/lab17/short-complete.txt
```

Record: Complete Exec / 1 s: elapsed stop time, TERM receipt, presence/absence of cleanup completion, ExitCode, OOMKilled and marker-copy result.

Step 7. Remove the three stopped containers after recording evidence

Run in: VM terminal 1, same shell

```
docker rm ce-lab17-wrapper ce-lab17-exec ce-lab17-short
docker ps -a --filter 'name=~ce-lab17-' --format '{{.Names}}'
```

Record: Confirm all three evidence rows are complete before removal. Record whether the final command lists any remaining lab containers.

Recovery check: The exec/five-second case completed cleanup, the two forced cases have explicit evidence, and all three test containers are removed after evidence collection.

Write your answer

Use the observations recorded beside each step.

Case	Worker PID	TERM / cleanup	Exit / OOMKilled
Wrapper / 5 s			
Exec / 5 s			

Case	Worker PID	TERM / cleanup	Exit / OOMKilled
Exec / 1 s			

1. Compare the wrapper and exec cases with the same five-second stop budget. What worker PID, TERM receipt, cleanup marker, exit code and OOM flag did each produce?
2. Compare the two exec cases. Which evidence separates successful signal delivery from enough time to finish cleanup?
3. Explain the cause of each forced exit and show that the test containers were removed after recording their evidence.

Apply the same reasoning

Would increasing the wrapper's stop budget to 30 seconds repair its missing signal forwarding?

Save results before removing this lab's resources and files: `./lab.sh 17 reset`

Lab 18 - Persistence, ownership and read-only mounts

Is a failed container write caused by lost data, Unix permissions or a read-only mount?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 18 setup` on the host. No other lab is required.

Theory you need

A container's writable layer belongs to that container. Stop and start preserve it; removing and replacing the container does not. A named volume has a separate lifetime and remains when a container using it is removed. Persistence is not a backup or a guarantee against deleting the volume.

Linux permissions compare numeric UIDs and GIDs, not account names. Here the volume directory starts owned by UID 0 with mode 0755. UID 10001 can read and traverse it but cannot create files because only the owner has directory write permission. Creating a file requires write and execute permission on its parent directory.

Correcting the directory owner to 10001 repairs this specific permission mismatch without making it writable by every user. This lab uses rootful Docker without user-namespace remapping; mappings and host security policy can add other permission checks in other environments.

A read-only volume mount independently prevents writes even when ownership is correct. `chmod` or `chown` cannot override that mount property. Inspect the mount's RW field and compare the exact error; Permission denied and Read-only file system point to different layers.

Mounts can hide files that exist at the same path in the image. A missing file therefore does not always mean deletion. This fixture uses a dedicated `/data` path and deliberately compares a volume file with `/ephemeral.txt` outside that mount.

Source: docs/chaos-theory.md: §§5.13.3-5.13.4.

Experiment

Use only the named volume `ce-lab18-data` and disposable `ce-lab18` containers. Setup resets this exercise's data; save observations elsewhere.

Before running: Predict which file survives replacement and whether UID 10001 can write before chown, after chown and with a read-only mount.

Measurement key

Signal	Meaning
UID / GID / directory mode	id and numeric ls output show the identity and permissions used by the kernel, even if no account name exists.
Mounts / RW	Docker inspect identifies the named volume and whether this container mounted it writable. Volume existence alone does not prove write access.
ephemeral.txt / persisted.txt	Compare paths after replacing the container, not merely restarting the original one.
write exit / error	Read the failing command result directly. A successful later ls must not hide the earlier write failure.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 18 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Create one file in each storage layer

Run in: VM terminal 1

```
docker run --name ce-lab18-original --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'echo layer > /ephemeral.txt; echo volume > /data/persisted.txt; cat /ephemeral.txt /data/persisted.txt; ls
-l /data'
docker cp ce-lab18-original:/ephemeral.txt ~/labs/lab18/original-layer.txt
docker inspect -f '{{json .Mounts}}' ce-lab18-original
```

Record: Fill Original container: both file contents, /data owner and mode, volume name and mount RW. Keep the copied original-layer.txt as evidence from this container.

Step 2. Replace the container and check which file remains

Run in: VM terminal 1, same shell

```
docker rm ce-lab18-original
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'cat /data/persisted.txt; test ! -e /ephemeral.txt && echo writable-layer-file-absent; ls -ldn /data'
```

Record: Fill Replacement container: persisted.txt contents, the ephemeral.txt check result and /data ownership. Compare both file paths with the original container.

Step 3. Attempt the write as UID 10001 and diagnose the failure

Run in: VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

Record: Fill UID mismatch: process UID/GID, directory owner/mode, exact error and write exit. Compare the mount options with the previous case.

Step 4. Change only the directory owner and repeat the write

Run in: VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim chown 10001:10001 /data
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

Record: Fill Owner corrected: directory owner/mode and repeated write exit. Compare the identity, ownership, mode and mount options with the failed case.

Step 5. Change only the mount to read-only and inspect the failure

Run in: VM terminal 1, same shell

```
docker run --name ce-lab18-readonly --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data,readonly python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
docker inspect -f '{{json .Mounts}}' ce-lab18-readonly
```

Record: Fill Read-only mount: identity, directory owner/mode, exact error, write exit and Mounts.RW. Keep the stopped container until those values are recorded.

Step 6. Restore a writable mount and confirm both data and writes

Run in: VM terminal 1, same shell

```
docker run --name ce-lab18-recovered --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c 'ls -ldn /data; echo recovered > /data/user.txt && cat /data/user.txt /data/persisted.txt'
echo "write/read exit=$?"
docker inspect -f '{{json .Mounts}}' ce-lab18-recovered
```

Record: Fill Writable recovery: owner/mode, RW, command exit and both file contents. Compare the original volume data with its current contents.

Step 7. Remove the exercise containers and volume after recording results

Run in: VM terminal 1, same shell

```
docker rm ce-lab18-readonly ce-lab18-recovered
docker volume rm ce-lab18-data
docker ps -a --filter 'name=~/ce-lab18-' --format '{{.Names}}'
docker volume ls --format '{{.Name}}' | grep -x ce-lab18-data
```

Record: Confirm all six evidence rows are filled before removal. Record any remaining lab containers or exact ce-lab18-data volume; no grep match returns status 1.

Recovery check: A write succeeds again after restoring a writable mount, and the dedicated test volume is removed only after evidence has been recorded.

Write your answer

Use the observations recorded beside each step.

Phase	Files visible	Owner / mount RW	Write result
Original container			
Replacement container			
UID mismatch			
Owner corrected			
Read-only mount			
Writable recovery			

1. Which file survives container replacement? Use the original and replacement observations to distinguish a named volume from the container writable layer.
2. Why does UID 10001 initially fail to write, and which exact ownership change repairs it? Cite numeric ownership, mode and write status.
3. Why does the same user fail through the read-only mount? Compare the error and RW field, then prove writable recovery and data persistence.

Apply the same reasoning

Would `chmod 777` repair a read-only mount, and would it be an appropriate first response to an ownership mismatch?

Save results before removing this lab's resources and files: `./lab.sh 18 reset`

Lab 19 - Unscheduled Pods versus memory-killed containers

Did the workload fail before placement or after exceeding its memory limit?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 19 setup` on the host. No other lab is required.

Theory you need

Requests and limits answer different questions. The scheduler accounts for requested resources against node allocatable capacity and existing requests. It does not place a Pod merely because live CPU usage looks low. CPU 20m means 0.02 CPU; CPU 1000 means one thousand CPUs, not one thousand millicores.

An impossible request leaves this Pod unassigned with `PodScheduled=False` and reason `Unschedulable`. No application container has started, so application logs cannot diagnose its execution. Scheduling events describe constraints; inspect the current Pod UID because a previous object may have used the same name.

A memory limit is enforced at runtime through the container's memory cgroup. Kubernetes Mi means MiB (1048576 bytes). The OOM fixture requests 16 MiB but has a 32 MiB limit and deliberately touches 96 MiB. Its request fits; its allocation does not. A memory request is not the maximum amount the application may use.

`OOMKilled` in `lastState.terminated`, an exit code and a rising `restartCount` identify a different stage from `Unschedulable`. Exit 137 alone cannot identify OOM. `CrashLoopBackOff` describes a delay before repeated restarts, not their root cause. Inspect the previous instance's logs while the Pod still exists.

These standalone Pods use `restartPolicy Always` by default. Kubelet can restart a container inside the same Pod UID. The procedure intentionally deletes and recreates the Pod between configurations because scheduling and resource fields cannot all be edited in place. That intentional UID change is not evidence of self-healing by a Deployment.

Repair the demonstrated cause. Restarting cannot make a 1000-CPU request fit. Repeated restarts cannot make the same 96 MiB allocation fit a 32 MiB limit. In a real workload, investigate memory growth before choosing a measured limit; simply removing limits can move the failure to the node.

Source: docs/chaos-theory.md: §§10.6.1-10.6.2.

Experiment

Use namespace `ce-lab19` on this lab's chaos cluster. Compare a healthy HTTP process, an impossible CPU request and a bounded memory allocation; then restore the healthy fixture.

Before running: Predict node assignment, available logs and restart evidence for the impossible request and the memory limit failure.

Measurement key

Signal	Meaning
nodeName / PodScheduled / events	Distinguish no placement from a node-assigned runtime problem. Events are filtered by this Pod UID.
requests / limits / allocatable	Read configured quantities and node capacity. No metrics-server is required for this diagnosis.
lastState.terminated / restartCount	The previous container result and restart count remain attached to this Pod. Capture them before deleting it.
logs --previous	Reads the previous container instance, not a previous Pod with the same name. Missing logs are not proof that no failure happened.
k19	kubectl using Lab 19's private kubeconfig, kind-lab19 context and only namespace ce-lab19.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 19 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Record the healthy placement and configured resources

Run in: VM terminal 1

```
source ~/labs/lab19/helpers.sh
k19 get nodes -o
  custom-columns='NAME:.metadata.name,CPU:.status.allocatable.cpu,MEMORY:.status.allocatable.memory'
k19 get pod resource-web -o json | jq
  '{uid:.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,status:.status}'
k19 logs resource-web
```

Record: Fill Healthy: Pod UID, assigned node, Scheduled/Ready conditions, requests/limits and restart count. Record node allocatable capacity for the scheduling comparison.

Step 2. Replace the baseline with the impossible CPU request

Run in: VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/unscheduled.yaml
k19 wait pod/resource-web --for=jsonpath='{.status.conditions[?(@.type=="PodScheduled")].reason}'=Unschedulable
  --timeout=60s --request-timeout=0
k19 get pod resource-web -o json | jq
  '{uid:.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,conditions:.status.conditions}'
```

Record: Fill 1000 CPU request: UID, node value, CPU request and PodScheduled status/reason. Record the wait result and compare with the healthy Pod.

Step 3. Collect scheduling events and check application-log availability

Run in: VM terminal 1, same shell

```
uid=$(k19 get pod resource-web -o jsonpath='{.metadata.uid}')
k19 get events --field-selector "involvedObject.uid=$uid"
k19 logs resource-web --request-timeout=5s
echo "logs exit=$?"
```

Record: Add the current Pod UID, scheduling-event message, log-request result and exit to the 1000 CPU request row. Use these to identify the earliest failed stage.

Step 4. Replace it with the memory failure and wait for OOM evidence

Run in: VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/oom.yaml
oom_uid=$(k19 get pod resource-web -o jsonpath='{.metadata.uid}')
echo "OOM fixture initial UID=$oom_uid"
k19 wait pod/resource-web --for=jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'=OOMKilled
--timeout=90s --request-timeout=0
```

Record: Record the OOM fixture UID at creation and the wait result. Do not delete the Pod before the next evidence-collection step.

Step 5. Save the runtime crash, prior logs and current-UID events

Run in: VM terminal 1, same shell

```
k19 get pod resource-web -o json | tee ~/labs/lab19/oom-evidence.json | jq
'{"uid":.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,conditions:.status.conditions,containers:.status.containerStatuses}'
k19 logs resource-web --previous | tee ~/labs/lab19/previous.log
uid=$(k19 get pod resource-web -o jsonpath='{.metadata.uid}')
printf 'Initial OOM UID=%s; current UID=%s\n' "$oom_uid" "$uid"
k19 get events --field-selector "involvedObject.uid=$uid"
```

Record: Fill 32 MiB limit / 96 MiB allocation: node, initial/current OOM Pod UID, Scheduled/Ready, requests/limit, last termination reason/exit, restart count and previous log. Preserve both saved files.

Step 6. Restore the healthy fixture and verify recovery

Run in: VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/good.yaml
k19 wait pod/resource-web --for=condition=Ready --timeout=60s --request-timeout=0
k19 get pod resource-web -o json | jq
'{"uid":.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,conditions:.status.conditions,containers:.status.containerStatuses}'
```

Record: Fill Healthy restored: UID, node, Scheduled/Ready, resources, restart count and prior termination state. Compare the UID and resource settings with the earlier cases.

Recovery check: The healthy resource-web Pod is Ready, and its current container has no OOM termination or repeated restarts.

Write your answer

Use the observations recorded beside each step.

Phase	UID / node	Scheduled / Ready	Reason / restarts
Healthy			
1000 CPU request			
32 MiB limit / 96 MiB allocation			
Healthy restored			

1. Compare the healthy Pod with the 1000-CPU request: which stage fails, and what do node assignment, PodScheduled, current-UID events and logs show?

2. For the 32 MiB memory limit, record the request, deliberate allocation, node, UID, termination reason, exit and restart count. How does this differ from the scheduling failure?

3. Show that the restored Pod is Ready with the healthy resources. Explain which UID changes were intentional replacement and which restart happened within one Pod.

Apply the same reasoning

Would increasing the CPU limit fix an unschedulable Pod whose CPU request exceeds every node's capacity?

Save results before removing this lab's resources and files: `./lab.sh 19 reset`

Lab 20 - Trace DNS, Service ports and the application listener

At which layer does a failed in-cluster HTTP request break?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 20 setup` on the host. No other lab is required.

Theory you need

Follow the request through name resolution, Service IP and port, endpoint address and target port, then the application's listener. The Service selector matches Pod labels; EndpointSlices list those backend addresses, ports and readiness conditions. A DNS answer identifies an address; it does not establish that HTTP will succeed.

An ordinary ClusterIP Service has a DNS name scoped by namespace. `web.ce-lab20.svc.cluster.local` identifies this fixture in kind's `cluster.local` domain. A wrong namespace in the name can fail resolution while the correct Service IP still works. Diagnose that difference before changing the cluster DNS server.

Service port is the client-facing port; `targetPort` is the backend port. This Service accepts port 80 and forwards to 8080. Changing `targetPort` to 8099 keeps DNS, selectors and Pod readiness intact, but forwards to a port with no server. `containerPort` metadata does not open a socket or rewrite an application's listen port.

Compare HTTP by Service name, Service IP and direct Pod IP from the same client Pod. Name failure with IP success points to resolution or naming. Service failure with direct Pod success points toward Service selection, port mapping or routing; the endpoint and port evidence narrows the cause.

A listener bound to `127.0.0.1` accepts only connections inside that Pod's network namespace. It can answer a `localhost` request from `kubectl exec` while refusing the client Pod's request to its Pod IP. The HTTP readiness probe also uses the Pod IP by default, so this bind fault makes the server unready and removes ordinary Service eligibility.

A timeout is not a diagnosis of a NetworkPolicy or DNS failure. Inspect the layer actually tested. This fixture does not install a NetworkPolicy-enforcing CNI and makes no NetworkPolicy claims. Other real causes include application failure, firewall rules and CNI routing.

Source: `docs/chaos-theory.md`: §§10.7.1-10.7.2.

Experiment

Use namespace `ce-lab20` with one Python server and one Python client. Change the requested name, then `targetPort`, then the bind address; restore each fault before the next comparison.

Before running: Predict DNS, Service-IP HTTP, direct-Pod HTTP and readiness for the wrong name, wrong `targetPort` and loopback-only listener.

Measurement key

Signal	Meaning
resolve / probe	Helpers run socket DNS lookup or a bounded HTTP request inside the same client Pod. Errors return nonzero and are printed, not counted as healthy HTTP.
Service ports / EndpointSlices	Compare the requested Service port with the endpoint port and readiness. A nonempty endpoint list can still contain the wrong port.
localhost versus Pod IP	Local success proves a local listener; it does not prove that another Pod can reach that listener.
k20	kubectl fixed to kind-lab20 and ce-lab20. The client remains unchanged while the server is intentionally recreated for the bind comparison.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 20 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Load the client helpers and bounded endpoint-port check

Run in: VM terminal 1

```
source ~/labs/lab20/helpers.sh
wait_endpoint_port() {
  local port=$1 endpoints i
  for i in $(seq 1 30); do
    if endpoints=$(k20 get endpointslices -l kubernetes.io/service-name=web -o json) &&
      printf '%s' "$endpoints" | jq -e --argjson port "$port" '(.items | length) > 0 and all(.items[];
any(.ports[]; .port == $port))' >/dev/null; then
      return 0
    fi
    sleep 1
  done
  echo "Endpoint port did not become $port; stop and inspect the Service and EndpointSlices." >&2
  return 1
}
```

Record: Use this VM shell for every step so `k20`, `resolve`, `probe` and `wait_endpoint_port` remain defined. The port check reads the current EndpointSlice list for up to 30 attempts.

Step 2. Measure the healthy request path from the client Pod

Run in: VM terminal 1, same shell

```
SVC_IP=$(k20 get svc web -o jsonpath='{.spec.clusterIP}')
POD_IP=$(k20 get pod web -o jsonpath='{.status.podIP}')
printf 'Service IP=%s; Pod IP=%s\n' "$SVC_IP" "$POD_IP"
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k20 get pod web
k20 get svc web -o json | jq '{selector:.spec.selector,ports:.spec.ports}'
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
```

Record: Fill Baseline: resolved address, each HTTP result, Service/Pod IPs, Pod Ready status, Service port/targetPort and endpoint port/readiness. All helpers use the same client Pod.

Step 3. Request the wrong namespace name, then retry the correct path

Run in: VM terminal 1, same shell

```
resolve web.ce-lab20-missing.svc.cluster.local
echo "wrong-name DNS exit=$?"
probe http://web.ce-lab20-missing.svc.cluster.local/
echo "wrong-name HTTP exit=$?"
probe "http://$SVC_IP/"
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
```

Record: Fill Wrong namespace name: DNS/HTTP errors and exits, correct Service-IP result and correct-name recovery result. No cluster setting changed.

Step 4. Change targetPort to 8099 and compare Service with direct HTTP

Run in: VM terminal 1, same shell

```
k20 patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8099}] '
wait_endpoint_port 8099
resolve web.ce-lab20.svc.cluster.local
k20 get pod web
k20 get svc web -o json | jq '.spec.ports'
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
```

Record: Fill Wrong targetPort: DNS, Service-name/IP and direct-Pod HTTP results, Pod readiness and endpoint port. Wait for the endpoint port change before interpreting traffic.

Step 5. Restore targetPort to 8080 before the next fault

Run in: VM terminal 1, same shell

```
k20 patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8080}] '
wait_endpoint_port 8080
for i in $(seq 1 10); do probe http://web.ce-lab20.svc.cluster.local/ && break; sleep 1; done
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
```

Record: Fill Port restored: final Service-name/IP and direct-Pod HTTP results plus endpoint port/readiness. Continue only after the final Service request succeeds.

Step 6. Replace only the server bind address and compare local with remote

Run in: VM terminal 1, same shell

```
k20 delete pod web --wait=true --timeout=30s
k20 apply -f ~/labs/lab20/loopback.yaml
k20 wait pod/web --for=jsonpath='{.status.phase}'=Running --timeout=60s --request-timeout=0
POD_IP=$(k20 get pod web -o jsonpath='{.status.podIP}')
for i in $(seq 1 10); do k20 exec web -- python -c 'import urllib.request;
    print(urllib.request.urlopen("http://127.0.0.1:8080/",timeout=3).status)' && break; sleep 1; done
k20 exec web -- python -c 'import urllib.request;
    print(urllib.request.urlopen("http://127.0.0.1:8080/",timeout=3).status)'
resolve web.ce-lab20.svc.cluster.local
probe "http://$POD_IP:8080/"
k20 get pod web -o json | jq
' {ip:.status.podIP,command:.spec.containers[0].command,conditions:.status.conditions} '
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
probe http://web.ce-lab20.svc.cluster.local/
```

Record: Fill Loopback listener: Pod IP, bind address, localhost result, DNS result, remote Pod/Service results, Pod Ready condition and endpoint readiness.

Step 7. Restore the listener and check every path again

Run in: VM terminal 1, same shell

```
k20 delete pod web --wait=true --timeout=30s
k20 apply -f ~/labs/lab20/server.yaml
k20 wait pod/web --for=condition=Ready --timeout=60s --request-timeout=0
POD_IP=$(k20 get pod web -o jsonpath='{.status.podIP}')
for i in $(seq 1 10); do probe http://web.ce-lab20.svc.cluster.local/ && break; sleep 1; done
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k20 get pod web -o json | jq
  '{ip:.status.podIP,command:.spec.containers[0].command,conditions:.status.conditions}'
k20 get svc web -o json | jq '.spec.ports'
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
```

Record: Fill Listener restored: bind address, Pod IP, DNS/three HTTP results, Ready condition, targetPort and endpoint readiness. Compare with the initial baseline.

Recovery check: DNS and Service HTTP work, targetPort is 8080, and the server listens on 0.0.0.0 with a Ready endpoint.

Write your answer

Use the observations recorded beside each step.

Phase	DNS	Service / Pod HTTP	Ready / endpoint port
Baseline			
Wrong namespace name			
Wrong targetPort			
Port restored			
Loopback listener			
Listener restored			

1. Record the healthy DNS, Service-name HTTP, Service-IP HTTP, direct-Pod HTTP, readiness and endpoint port. Which of these paths does each probe test?
2. For the wrong namespace name, which requests fail and which still work? Prove recovery by using the correct name.
3. For the wrong targetPort, compare DNS, Service and direct-Pod results with the endpoint port. Show the result after restoring 8080.
4. For the loopback-only listener, compare localhost, remote Pod-IP and Service HTTP with readiness. Show the full path working after listener recovery.

Apply the same reasoning

If DNS works but direct Pod-IP HTTP also fails, is changing only the Service selector a justified repair?

Save results before removing this lab's resources and files: `./lab.sh 20 reset`

Lab 21 - Configuration freshness and a stalled rollout

Why can a ConfigMap update leave old behavior running, and why can a rollout fail while the application stays available?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 21 setup` on the host. No other lab is required.

Theory you need

A ConfigMap is a separate API object. This application reads its message through an environment variable when its container starts. Updating the ConfigMap does not rewrite the environment of existing processes and does not change the Deployment's Pod template, so it does not itself create a rollout.

A projected ConfigMap volume behaves differently. Ordinary projected files can update eventually, but the application must reread or reload them; a subPath mount does not receive those updates. Do not infer environment refresh from file-projection behavior. This lab tests environment injection specifically.

A Deployment rollout occurs when its Pod template changes. rollout restart changes a template annotation and creates new Pods; they read the current ConfigMap. Record Pod UIDs and the actual HTTP response from each Pod so a mixture of old and new processes cannot hide behind a single sampled response.

This Deployment has two replicas, `maxUnavailable=0` and `maxSurge=1`. During this update it can add one Pod while keeping two replicas available. `minReadySeconds=2` requires a new Pod to remain Ready for two seconds before it counts as available. The strategy governs replacement; a `PodDisruptionBudget` does not control this rollout.

A wrong readiness port can leave a new process Running but unready. It must not replace an available old replica under this strategy. When no rollout progress occurs for the configured 30-second deadline, the Deployment reports `ProgressDeadlineExceeded`. Kubernetes reports the stall; it does not automatically undo the change.

Rollout history stores Pod-template revisions, not snapshots of separate ConfigMaps. Undo to the recorded good revision repairs the bad readiness port, but does not restore the ConfigMap's old message. Recovery requires checking both template and configuration, then confirming the intended responses and replica counts.

Source: docs/chaos-theory.md: §§10.8.1-10.8.2.

Experiment

Use namespace `ce-lab21`, two Python HTTP replicas, an environment-backed message and a deliberately wrong readiness port. Query every Pod directly through the API proxy; this measures application responses, not Service routing.

Before running: Predict Pod UIDs and response values after only the ConfigMap changes, after restart, during the bad probe rollout and after undo.

Measurement key

Signal	Meaning
ConfigMap value / per-Pod HTTP	Compare desired configuration with each running process. An updated object is not proof that every process consumed it.
UID / revision / ReplicaSets	UID changes show replacement. The Deployment revision identifies a Pod template; ReplicaSet desired/current/ready counts show rollout progress.
Available / Progressing	Availability of old replicas and progress of the new template are separate conditions. Both must be interpreted with the replica counts.
readiness port / restartCount	A bad readiness destination can block rollout without crashing or restarting the process.
k21 / responses	kubectl targets kind-lab21 and ce-lab21. responses prints each Pod name and its HTTP body; a missing or failed query returns nonzero.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 21 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Record the original configuration and every consumer

Run in: VM terminal 1

```
source ~/labs/lab21/helpers.sh
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k21 get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k21 get deployment config-web -o json | jq
  '{revision:.metadata.annotations["deployment.kubernetes.io/revision"],desired:.spec.replicas,ready:.status.r
  eadyReplicas,conditions:.status.conditions}'
responses
```

Record: Fill Version one: ConfigMap value, each Pod name/UID/Ready status and HTTP body, desired/ready counts, revision and conditions.

Step 2. Change only the ConfigMap and query the same processes

Run in: VM terminal 1, same shell

```
k21 patch configmap app-config --type=merge -p '{"data":{"message":"version-two"}}'
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k21 get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k21 get deployment config-web -o json | jq
  '{revision:.metadata.annotations["deployment.kubernetes.io/revision"],ready:.status.readyReplicas,conditions
  :.status.conditions}'
responses
```

Record: Fill ConfigMap version two only: ConfigMap value, Pod UIDs, each HTTP response, Ready count and revision. Compare every value with step 1.

Step 3. Restart consumers and save the good revision for undo

Run in: VM terminal 1, same shell

```
k21 rollout restart deployment/config-web
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
k21 get pods -l app=config-web -o
    custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
responses
stable_revision=$(k21 get deployment config-web -o json | jq -r
    '.metadata.annotations["deployment.kubernetes.io/revision"]')
printf '%s\n' "$stable_revision" | tee ~/labs/lab21/stable-revision.txt
k21 rollout history deployment/config-web
k21 get deployment config-web -o json | jq
    '{desired:.spec.replicas,ready:.status.readyReplicas,probePort:.spec.template.spec.containers[0].readinessPr
    obe.httpGet.port,conditions:.status.conditions}'
```

Record: Fill Restarted consumers: UIDs, each HTTP response, ConfigMap value, Ready count, conditions and the saved stable revision. Keep stable-revision.txt for step 5.

Step 4. Break the new readiness port and collect stalled-rollout evidence

Run in: VM terminal 1, same shell

```
k21 patch deployment config-web --type=json -p
    '[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","value":9999}]'
k21 wait deployment/config-web
    --for=jsonpath='{.status.conditions[?(@.type=="Progressing")].reason}'=ProgressDeadlineExceeded
    --timeout=90s --request-timeout=0
k21 rollout status deployment/config-web --timeout=5s --request-timeout=0
echo "rollout status exit=$?"
k21 get deployment config-web -o json | jq
    '{revision:.metadata.annotations["deployment.kubernetes.io/revision"],desired:.spec.replicas,status:.status}'
responses
k21 get rs -l app=config-web
k21 get pods -l app=config-web -o json | jq '.items[] |
    {name:.metadata.name,uid:.metadata.uid,probe:.spec.containers[0].readinessProbe,conditions:.status.condition
    s,restarts:.status.containerStatuses[0].restartCount}'
responses
```

Record: Fill Wrong readiness port: rollout result/exit, revision, desired/updated/ready/available counts, Available and Progressing conditions, each Pod UID/probe/Ready/restarts and direct HTTP body.

Step 5. Undo to the saved template and inspect configuration separately

Run in: VM terminal 1, same shell

```
stable_revision=$(cat ~/labs/lab21/stable-revision.txt)
k21 rollout undo deployment/config-web --to-revision="$stable_revision"
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k21 get pods -l app=config-web -o
    custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
responses
k21 get deployment config-web -o json | jq
    '{revision:.metadata.annotations["deployment.kubernetes.io/revision"],probePort:.spec.template.spec.containe
    rs[0].readinessProbe.httpGet.port,desired:.spec.replicas,ready:.status.readyReplicas,conditions:.status.cond
    itions}'
```

Record: Fill Undo to stable revision: resulting revision, probe port, ConfigMap value, Pod UIDs/responses, Ready count and rollout conditions. Compare the template and ConfigMap separately.

Step 6. Restore version-one and verify configuration plus all consumers

Run in: VM terminal 1, same shell

```
k21 patch configmap app-config --type=merge -p '{"data":{"message":"version-one"}}'
k21 rollout restart deployment/config-web
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
k21 get configmap app-config -o jsonpath='{.data.message}'{"\n"}'
k21 get pods -l app=config-web -o
    custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
responses
k21 get deployment config-web -o json | jq
    '{revision:.metadata.annotations["deployment.kubernetes.io/revision"],probePort:.spec.template.spec.container
    rs[0].readinessProbe.httpGet.port,desired:.spec.replicas,ready:.status.readyReplicas,conditions:.status.conditions}'
```

Record: Fill Version one restored: ConfigMap value, UIDs, both response bodies, probe port, desired/Ready count, revision and conditions. Compare every current consumer with the intended configuration.

Recovery check: The ConfigMap says version-one, exactly two Pods are Ready, both HTTP responses say version-one, and the readiness port is 8080.

Write your answer

Use the observations recorded beside each step.

Phase	ConfigMap / responses	UIDs / Ready replicas	Revision / conditions
Version one			
ConfigMap version two only			
Restarted consumers			
Wrong readiness port			
Undo to stable revision			
Version one restored			

1. Compare the ConfigMap, Pod UIDs and per-Pod HTTP responses before and after changing only the ConfigMap. Did the running processes consume the new value?
2. After rollout restart, record the UIDs, responses and stable template revision. What operation made configuration take effect?
3. For the bad readiness port, explain the new Pod state, old-replica availability, replica counts and Progressing condition. Does direct application HTTP still work?
4. After undo, what did the template restore and what configuration stayed changed? Show the separate actions and evidence that return every consumer to version-one.

Apply the same reasoning

Would undo to the original Pod-template revision necessarily restore the original environment value from the ConfigMap?

Save results before removing this lab's resources and files: `./lab.sh 21 reset`

Solutions

Compare after recording your own results. The expected patterns below assume a working baseline and a confirmed injection. Differences require analysis; do not replace observations with the expected answer.

Lab 00 - Prepare the learning environment

1. Which kernel, cgroup, Docker, container-execution and disk checks passed? Give their observed values.

Report the actual checks. The required cgroup results are cgroup2fs and Docker cgroups=2; hello-world must run successfully and the root filesystem must have at least 60 GiB total capacity. A failed check means this environment is not ready for the later experiments.

2. Which source commit, tool versions and image identities did you save, and where is the baseline record?

The record is ~/labs/lab00/baseline.txt. Cite its actual tool versions, source commit, local image IDs and registry digests so another run can be compared with this environment.

3. Why must you fix a failed baseline before injecting a fault?

A fault comparison requires a working no-fault baseline. If the same check already fails, a later failure does not establish that the injection caused it.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check the kernel, cgroup, Docker and disk baseline: cgroup2fs, Docker cgroups=2, a successful hello-world run, and at least 60 GiB total root capacity.

Step 2. Record the source commit, versions and image digests: The book source commit is 3e3ee64db71f51a5e9f79af8562dd4aa913a0e71. Each listed image has a local ID and a registry digest; an inspection error is a missing prerequisite.

Step 3. Save the baseline record: baseline.txt contains the timestamp, all baseline checks and the recorded environment identities.

Apply the same reasoning: hello-world checks shared container tooling. It does not test the Redis service, its listener or the client path used by Lab 01.

Lab 01 - A timeout makes a silent failure manageable

1. Did the baseline and recovered requests succeed? What proves the firewall fault was removed?

Baseline and recovery should both print PONG and OK True. The final firewall search should find no ce-lab01 rule (grep status 1). Cite your observed output; an unsuccessful recovery leaves the comparison incomplete.

2. For REJECT, DROP and DROP with TIMEOUT=0.5, did the handler run, what ended the wait, and what were the elapsed time and exit status? Cite each rule's packet counter.

REJECT should print DEGRADED ConnectionError promptly and client exit=0 because the error is caught. DROP without an application timeout should reach the five-second watchdog, print client exit=124, and have no handler or elapsed line. DROP with TIMEOUT=0.5 should print DEGRADED TimeoutError near 0.5 seconds and client exit=0. Report actual timings and nonzero packet counters for all three cases; zero matched packets do not demonstrate the intended fault.

3. Why does try/except need an application timeout to handle a silent dependency promptly?

try/except reacts only after the dependency call raises an error. A silent DROP can leave that call waiting beyond the experiment's watchdog. The client's connection/read timeout makes the call raise an error the handler can catch; the external watchdog terminates the process without running its fallback.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check the Redis baseline: PONG, then OK True, an elapsed time and client exit=0. Stop if this baseline fails.

Step 2. Case 1 - REJECT without an application timeout: DEGRADED ConnectionError, a short elapsed time, client exit=0 and a nonzero REJECT packet counter. The runner removes its rule before returning.

Step 3. Case 2 - DROP without an application timeout: The external watchdog stops the client after five seconds with client exit=124. There should be no DEGRADED or elapsed line; the DROP rule must have matched packets.

Step 4. Case 3 - DROP with a 0.5-second application timeout: DEGRADED TimeoutError near 0.5 seconds, client exit=0 and a nonzero DROP packet counter. The application handles its own timeout before the watchdog fires.

Step 5. Prove Redis and firewall recovery: PONG and OK True with client exit=0; the firewall search prints no rule and grep exit=1.

Apply the same reasoning: No. Attempt budgets and any backoff accumulate. A whole-request deadline must also cover retries and other work.

Lab 02 - Prove why a process was killed

1. What exit status did the deliberate SIGKILL produce, and how do you know its cause?

Bash should report 137 for the deliberate SIGKILL. The command explicitly sends signal 9 to its own PID, so its cause is known without inferring it from the status.

2. What terminated the memory-limited allocator? Cite its PID, experiment time, unit result and matching kernel or systemd-oomd evidence.

The allocator should exceed its 128 MiB cgroup limit before the 20-second runtime bound. A unit result of oom-kill together with a matching kernel memory-cgroup kill identifies a kernel OOM kill. Match the allocator PID and the saved time window. If the evidence shows timeout or a matching systemd-oomd action instead, report that cause; missing evidence leaves the diagnosis unproven.

3. Why can neither exit status 137 nor the systemd-run exit status diagnose an OOM kill by itself?

137 is consistent with SIGKILL but does not explain who sent it or why; a program can also exit 137 explicitly. systemd-run reports the unit outcome and need not return the child's Bash exit status. The unit and matching cause evidence supply the diagnosis.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Confirm memory controls and begin an evidence record: The memory controller is available and no previous ce-lab02 allocator is active. Stop if either prerequisite is missing.

Step 2. Record a deliberate SIGKILL: Bash normally reports 137. Here you know the cause because the program deliberately sent SIGKILL.

Step 3. Run the allocator within its memory and time limits: The unit should fail before its 20-second runtime bound. Its nonzero command status alone does not identify the cause; the next step collects the evidence.

Step 4. Match the allocator termination to unit, kernel and userspace logs: A kernel OOM diagnosis requires the unit's oom-kill result and matching allocator PID in the kernel log. A matching oomd action or runtime timeout gives a different diagnosis.

Step 5. Stop the allocator if it remains active: ce-lab02 is inactive or already collected; no allocator remains active.

Apply the same reasoning: No. The allocator is constrained by its cgroup boundary. VM-wide availability and the cgroup's remaining allowance are different quantities.

Lab 03 - Restart policies have limits

1. How did A's PID, state and restart count change after one crash?

After one SIGKILL, A should become active again with a different MainPID and a higher NRestarts value. These observations establish replacement of the process; use your measured values.

2. What happened to A during the rapid crash burst? Cite the unit result and the matching start-limit journal message.

The burst should exhaust the four-start allowance within 20 seconds, leaving A failed with a start-limit result and a matching journal message. Restart=always requests another start but cannot override the rate limiter. If the limit was not reached, report the observed state and timing instead.

3. What did the proxy return in each phase, and why does that not establish A's health?

The proxy may keep returning HTTP 200 because B remains available. Cite each actual HTTP result. A shared response body does not identify which backend answered, so inspect A independently.

4. Which recovery commands restored A, and what proves both backends and the proxy recovered?

reset-failed clears the failed state and start counter; start requests a new process. Both backends should report active and the proxy should return lab03 OK. Record any failure to recover.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check baseline and clear the previous start counter: Both backends are active, A has a nonzero PID, Restart=always is set, and the proxy returns lab03 OK with HTTP 200. Stop if this baseline fails.

Step 2. Kill A once and inspect its recovery: A should return to active. A successful proxy response alone cannot prove that A restarted.

Step 3. Repeat crashes rapidly and inspect the refused restart: Look for "Start request repeated too quickly" and a failed unit. Result and restart counts can vary with timing and systemd version; check the journal.

Step 4. Recover and prove the proxy works again: Both backends are active and the proxy returns lab03 OK with HTTP 200 after the listeners recover.

Apply the same reasoning: It would test whether the rate limit caused the supervisor to give up. The application would still crash, and rapid restarts would still consume resources and need monitoring.

Lab 04 - CPU contention and a quota

1. What were the mean and range of iteration times at baseline, with the unrestricted neighbour, with the 20% neighbour and after recovery?

Use the printed mean and range for all four phases. The unrestricted neighbour should increase iteration time; the capped neighbour should reduce that increase. There is no fixed required speedup, and small differences within baseline variation do not establish a clear effect.

2. Was the neighbour active throughout each loaded measurement? Which measurements, if any, are inconclusive?

Both loaded measurements require an active neighbour before and after work.py. If the neighbour ended early, that phase mixes contention and recovery and is inconclusive.

3. What quota/period ratio did `cpu.max` show, and how much did the throttling counters increase during the capped measurement?

`cpu.max` should have `quota/period = 0.2`. Subtract the before counters from the after counters; rising `nr_throttled` and `throttled_usec` show the competitor used its budget and was throttled.

4. How do the timings, throttling evidence and recovered baseline explain the effect of capping the competitor?

The two jobs share one CPU. Restricting the competitor's CPU time should leave more scheduling time for the measured job. Quota enforcement plus shorter measured times and recovery toward baseline support that explanation; PSI and `vmstat` are supporting VM-wide signals.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Choose an allowed CPU: One CPU permitted by this shell's affinity; use the same shell and CPU for every phase.

Step 2. Record the baseline three times: Three runs produce 18 finite iteration timings with no deliberate competitor.

Step 3. Add one bounded CPU neighbour on the same CPU: Compare iteration time with baseline. PSI and `vmstat` provide supporting waiting measurements; neither alone proves which task caused the slowdown.

Step 4. Cap the neighbour at 20 percent and read throttling counters: `cpu.max` should show a quota/period ratio of 0.2, and `cpu.stat` should show throttling. If the neighbour ends before measurement finishes, this phase is inconclusive.

Step 5. Recover and repeat the baseline: The neighbour is no longer active. Recovered timings should move toward the baseline range; report measured variation rather than assuming exact equality.

Apply the same reasoning: No. It changes placement and removes the intended competition for a single CPU. Keep placement fixed when evaluating the quota.

Lab 05 - A container is several Linux mechanisms

1. What filesystem and PID views did the sandbox show, and which mechanisms create those views?

The sandbox should show the exported BusyBox tree under `/` and its shell as PID 1 with only sandbox processes in `ps`. `chroot` changes its filesystem root; the PID namespace and fresh `proc` mount provide the process view. The VM still sees the sandbox under an ordinary host PID.

2. What CPU, memory and task limits did the host read from the sandbox's cgroup?

`cpu.max` should have `quota/period = 0.2`, `memory.max` should be 134217728 bytes, and `pids.max` should be 50. Cite the actual values; these cgroup settings are independent of the filesystem and PID views.

3. How much did `nr_throttled` and `throttled_usec` increase while the busy loop ran? What does that prove beyond the configured quota?

Subtract the first `cpu.stat` snapshot from the second. Increasing `nr_throttled` and `throttled_usec` while the loop runs show active enforcement. A configured cap or an old nonzero total alone does not prove throttling during this measurement.

4. Why does an isolated process listing alone tell you nothing about the sandbox's CPU budget?

Namespaces control visibility while cgroups control resource budgets. A separate PID view can exist with or without a CPU cap; the sandbox still shares the VM's kernel and, here, its network.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Confirm the exported BusyBox root filesystem: The exported tree contains `bin` and `proc`, and no previous sandbox scope is active.

Step 2. Start the sandbox under a resource-limited systemd scope (this shell becomes the sandbox): Terminal 1 presents the BusyBox shell and stays inside the sandbox until you run exit.

Step 3. Inspect the view inside the sandbox: `$$` is 1, `ps` lists only the sandbox processes, and `/` is the BusyBox tree.

Step 4. Read the limits and PID namespaces from the host: `cpu.max 20000 100000`, `memory.max 134217728`, `pids.max 50`; `lsns` shows a separate PID namespace whose command is `sh`.

Step 5. Run a finite busy loop inside the sandbox: The loop runs for at most 30 seconds before timeout terminates it and the sandbox prompt returns. While it runs, switch immediately to VM terminal 2 and run the next step.

Step 6. Observe enforcement from the host while the loop runs: `nr_throttled` and `throttled_usec` increase during the loop. `top` may show the busy `sh` process near 20 percent CPU; the `cgroup` counter differences provide the direct enforcement evidence.

Step 7. After the loop finishes, exit the sandbox: After the finite loop ends, `exit` closes the BusyBox shell and returns terminal 1 to the VM shell.

Step 8. Confirm the scope stopped: The scope is inactive or already collected after the sandbox exits.

Apply the same reasoning: No. The PID view is controlled by the PID namespace and `proc` mount.

Removing a bandwidth limit changes resource consumption, not visibility.

Lab 06 - Separate containers can share a full filesystem

1. What did the identical one-MiB write return on the empty filesystem, while full, and after space was freed? Give the write status and available capacity for each phase.

The one-MiB probe should succeed before filling, fail while the shared mount is full, and succeed after the files consuming space are removed. Cite the actual exit statuses and `df` values for all phases.

2. What error and capacity measurements connect the failed write to shared-space exhaustion?

The filling write should reach No space left on device and `df` should show no available space on the 32 MiB `tmpfs`s. The later probe should report the same capacity error. A permission error or a different mount does not establish this explanation.

3. Why did a fresh container still fail, and which action restored its ability to write?

Each `docker run` creates a fresh container but bind-mounts the same `tmpfs`s. Removing the writer container leaves its file on that shared filesystem. Deleting the filling file restores capacity; the successful recovery probe demonstrates that shared-space exhaustion caused the failure.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Mount and identify the bounded shared filesystem: This exact lab path is a `tmpfs` with 32 MiB total capacity. Stop if the mount failed or a previous experiment remains.

Step 2. Baseline - write one MiB before filling: The one-MiB write succeeds before space is consumed.

Step 3. Fill only the 32 MiB `tmpfs`s from one container: The 40 MiB attempt exceeds the 32 MiB mount, `dd` reports No space left on device and `df` shows no available space. The printed `dd` status is the write's status; `df` may leave the container status zero.

Step 4. Repeat the identical probe from a fresh container while full: This new container's one-MiB write also fails with No space left on device and a nonzero status.

Step 5. Free capacity and repeat the identical write: The same probe succeeds after capacity is freed.

Step 6. Remove the probe and unmount the temporary filesystem: The directory is no longer a mountpoint and no `ce-lab06` container remains.

Apply the same reasoning: Its file is stored in the host bind mount, outside the disposable container layer. Remove that file or the temporary filesystem to release the capacity.

Lab 07 - Network delay accumulates across sequential calls

1. What were the mean and range for one and four exchanges at baseline, with the 1 ms control, with 25 ms delay, and after removal?

Report your observed mean, minimum and maximum for both exchange counts in each of the four phases. The 1 ms control should add a small delay, with timing noise; it is not a zero-cost control.

2. After subtracting each exchange count's own baseline, how do the 25 ms results compare with $N \times 25$ ms?

For the 25 ms phase, subtract the baseline for one exchange from its delayed mean and do the same independently for four exchanges. Expected additions are roughly 25 ms and 100 ms because each reply is another sequential wait. Report deviations from these estimates rather than replacing measured values.

3. Which qdisc configuration and counter changes prove the intended delay was applied to the measured traffic?

netem delay 1ms and then 25ms should appear only on ce-lab07's eth0. Increasing Sent packet/byte counters between the before and after snapshots establish that measured traffic crossed the queue; configuration alone does not.

4. Did exiting tc remove the delay? Which command removed it, and what measurements demonstrate recovery?

The qdisc remains after the installing tc command exits. `net qdisc del dev eth0 root` removes it. Recovery requires both the absence of netem and echo timings moving toward each case's own baseline.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Measure the unchanged dependency: The client receives every echo; eth0 has its default noqueue, with no custom root qdisc. Stop if either check fails.

Step 2. Measure a 1 ms control: netem remains installed after tc exits, with traffic counters. Added time should be small; record the range.

Step 3. Change only the dose to 25 ms: Four sequential exchanges should accumulate more delay than one. Compare baseline-subtracted times, not raw ratios.

Step 4. Remove the rule and repeat both probes: No netem remains and timing returns toward baseline.

Step 5. Calculate added time from each case's own baseline: Added time at the 25 ms dose should be roughly 25 ms for one exchange and 100 ms for four, with packet and scheduling variation. Recovery deltas should approach zero.

Apply the same reasoning: No. Sequential dependency waits add along a request's critical path. Independent parallel waits can overlap; other overhead still needs measurement.

Lab 08 - A syscall error tests application handling

1. What did the normal trace show, and did its fsync error stop the server?

A normal request should show writes followed by fsync and close. This fixture ignores the socket fsync EINVAL error and continues serving; an error line alone does not identify an outage cause.

2. For the injected request, connect the close return value, server log and process exit status.

The injected close should show -1 EIO and INJECTED. The matching "error closing socket" log and exit status 1 identify the application decision to exit. If the marker or exit was not observed, report the missing evidence instead of assuming the error path ran.

3. Compare HTTP status and response bytes for the triggering request, next request and restarted server. Why can they differ?

The triggering request may receive HTTP 200 and body bytes before close fails, because writing precedes closing. After the server exits, the next request should show 000 and no body. Restart should restore HTTP 200. Use the recorded result for each request; a completed triggering response does not prove continued availability.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Start the server and save its PID: HTTP should return 200 and body bytes. If it does not, inspect server.log before continuing.

Step 2. Attach the normal tracer and leave it running: The tracer waits for server syscalls; it does not inject faults.

Step 3. Send one normal traced request: The request should succeed and the server should remain running.

Step 4. Read the normal trace after detaching: Many write calls, one fsync and one close per request. fsync on a socket returns -1 EINVAL, which is not an outage cause.

Step 5. Attach the close-error injector and leave it running: The next close is replaced by an EIO return; trace output is saved in injected-trace.txt.

Step 6. Trigger the fault and collect the process result: The injected close should show EIO and INJECTED. The server should log "error closing socket" and exit 1; the response may already have reached the client.

Step 7. Test the next request independently: After the server exits, expect connection failure, HTTP 000, no body and a nonzero curl exit.

Step 8. Restart, verify HTTP recovery and stop the server: HTTP responds after restart; the test server is then stopped.

Apply the same reasoning: No. strace can skip the real call and substitute an error. Real close errors have syscall-specific semantics. The supported conclusion is about this application's reaction to the injected return value.

Lab 09 - A syscall denial need not crash the process

1. Compare getpid results before and after installing the filter. Which errno accompanied the denied call?

Before filtering, a successful getpid returns a positive PID. After filtering, syscall returns -1 and errno=13 (EACCES), matching the rule. Only interpret errno after an error return.

2. Why can the program still print, and what does its exit status mean?

The filter denies getpid while allowing other syscalls, including writing output. Exit 0 means the expected denial was detected, not that the denied getpid succeeded. Exits 1-3 mean filter setup failed; exit 4 means the expected denial was absent.

3. What do a fresh direct getpid call and /proc/self/status show about filter scope?

A fresh process started by the original shell should obtain a positive PID. Its Seccomp state may reflect an inherited environmental policy, but it does not inherit a filter installed only in the exited demonstration child. The filter is not host-wide.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Run the filter and capture its exit status: Before filtering, getpid prints a positive PID. After filtering, getpid result=-1 errno=13 and exit=0. Exit 1-3 means filter setup failed; 4 means the expected denial did not occur.

Step 2. Start a fresh instance from the original shell: The fresh instance should obtain a positive PID before installing its own filter, then observe EACCES and exit 0 again. Seccomp can reflect a pre-existing inherited policy; the first child did not install a host-wide filter.

Apply the same reasoning: No. It frees the library context in userspace. The kernel filter remains installed; the demonstration's later getpid call tests exactly this.

Lab 10 - Ownership, disruption budgets and Service routing

1. Use ownerReferences to explain which controller replaces the deleted Pod and how the UID proves replacement.

The Pod is owned by a ReplicaSet, which is owned by the Deployment. Deletion leaves desired replicas at three, so the ReplicaSet creates a new Pod. A new name and UID establish replacement; the old UID should be absent after deletion completes.

2. Compare the eviction request and direct deletion. What does the PDB protect?

With three healthy Pods and minAvailable=3, disruptionsAllowed should be zero and eviction should be rejected for violating the budget. Direct DELETE bypasses that guard. Forbidden or a successful eviction invalidates this comparison and needs investigation.

3. Calculate failed samples out of 100 and sampled availability. What do these measurements establish?

For 100 completed probes, failures are non-200 rows and availability is $(100 - \text{failures}) / 100 \times 100\%$. Report the measured value. Zero failures means these probes saw none; replacement alone and unobserved time between probes do not establish uninterrupted service.

4. Compare Ready Pods, ready endpoints and HTTP with the wrong and restored Service selectors.

With the unmatched selector, Pods can remain Ready while ready Service endpoints fall to zero and HTTP fails. Restoring the selector should restore three ready endpoints and HTTP without a workload rollout. Repeat endpoint observations if updates are still converging.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Checkpoint the prepared chaos cluster: Three Ready replicas, matching endpoints and a working /healthz probe. The workload may list Pods but may not delete them.

Step 2. Trace the Pod, ReplicaSet and Deployment owners: The Pod owner should be a ReplicaSet whose owner is deployment/goldpinger.

Step 3. Create the disruption budget and attempt eviction: The eviction is refused because it would violate the budget; the original UID still exists. Stop if the error is Forbidden or if eviction succeeds, since those do not establish the intended comparison.

Step 4. Start 100 probes about 0.2 s apart (start this right before the deletion): Each probes.log row contains a timestamp and HTTP code. A 000 means no HTTP status was received.

Step 5. Directly delete the same Pod despite the PDB; observe replacement for up to 60 seconds: Direct deletion should be accepted despite the PDB and a new Pod should appear. Exit 124 only ends the timed watch.

Step 6. After all 100 probes finish in terminal 2, collect recovery evidence: The deleted UID should be absent and three Ready replicas restored. Use the actual failed-sample count; zero failures only describes these probes.

Step 7. Break only Service selection and compare all three signals: Ready Pods can coexist with zero eligible Service endpoints and a failed request. Recheck endpoints if updates are still converging.

Step 8. Restore the selector and confirm routing recovery: Three eligible endpoints and successful HTTP return without a workload rollout.

Apply the same reasoning: No. Replica count does not guarantee placement across failure domains. This experiment deletes one Pod; it does not test node or zone loss.

Lab 11 - Health depends on what you test

1. Compare peer health and latency at TCP / 0, 100 and 400 ms. Explain any disagreement with TCP readiness.

With adequate timing margin, 100 ms should increase peer latency while remaining within the 300 ms budget. At 400 ms peer pings should time out, while TCP readiness can still succeed because the proxy accepts a connection before forwarding delayed data. Record the actual result from each peer and use fresh PingTime values.

2. Compare HTTP / 0, 400 and 1400 ms. How do the 300 ms peer budget and one-second readiness budget explain the results?

At HTTP / 0 both checks should succeed. At 400 ms peers can miss their 300 ms budget while HTTP readiness remains within one second. At 1400 ms both budgets are exceeded; readiness changes after consecutive failed checks. Missing or stale peer reports do not establish a timed-out request.

3. What happened to the slow Pod endpoint, and can peers still discover its IP directly?

When HTTP readiness fails, the slow Pod endpoint should become ineligible for ordinary Service routing. Baseline replicas may keep the Service available. Goldpinger discovers labelled Pod IPs directly, so an unready endpoint does not necessarily stop direct peer pings.

4. Use HTTP / removed, UID and restart counts to distinguish readiness recovery from restarting or replacing the Pod.

Removing the toxic should restore fresh peer success and Ready/endpoint readiness with the same HTTP-comparison UID and restart counts. A changed UID between TCP and HTTP baselines was intentional recreation. A change during fault removal would require a different explanation than readiness-only recovery.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Connect the proxy and record TCP readiness with no delay: The extra replica is Ready and peer reports for its IP show OK true. Establish this baseline before adding delay.

Step 2. Set 100 ms delay and record the TCP comparison: With enough margin, peer pings remain healthy but take longer. Readiness should remain true; record actual responses.

Step 3. Set 400 ms delay and record the TCP comparison: Look for failed peer pings while the Pod remains Ready. The 400 ms delay exceeds the 300 ms ping budget; readiness only checks TCP connection establishment.

Step 4. Switch only the extra Pod to HTTP readiness and record its baseline: Record a fresh UID for the HTTP configuration and a healthy baseline before injecting again.

Step 5. Set 400 ms delay and record the HTTP comparison: Peer requests can fail at 300 ms while HTTP readiness still succeeds within one second.

Step 6. Set 1400 ms delay and watch HTTP readiness change: The slow Pod becomes unready and its endpoint is not ready. UID and restart counts stay unchanged; other Service endpoints remain available.

Step 7. Remove the toxic and record recovery in the same Pod: The toxic list should be empty, fresh peer pings should recover, and Ready/endpoint readiness should return without a UID or restart-count change.

Step 8. Stop the port-forward and remove the extra Pod: The baseline Deployment should finish its rollout with three Ready replicas.

Apply the same reasoning: No. A healthy process can fail a dependency-based liveness check and restart repeatedly without repairing that dependency. Readiness controls routing; liveness should detect a condition a restart can plausibly fix.

Lab 12 - Locate startup failures and measure recovery

1. Complete all seven rows using fresh success/elapsed, Pod conditions and ready endpoints. Which stage failed in each deliberate fault?

A normal completed trial should meet the 30-second target. The delay trial should show an assigned but unready Pod; the impossible nodeSelector should show no node and Unschedulable; the selector fault can leave a Ready Pod with zero ready endpoints. Each should miss the HTTP deadline. Use the saved timestamped conditions, not success=0 alone, to locate the failed stage.

2. How do you distinguish a completed deadline miss, missing diagnostics and a checker execution error?

Exit 0 plus a fresh result with success=0 is a completed deadline miss. A diagnostic_error means that resource observation is unavailable even if timing completed. Nonzero checker exit means execution or cleanup failed; do not reuse old metrics as a new result. Compare trial timestamps and ce_last_run_timestamp_seconds to establish freshness.

3. Calculate the restored three-trial success fraction. What does this sample establish?

Divide successful restored trials by three completed restored trials; the supplied awk command calculates the fraction and percentage. Three successes give $3/3=1$, but report actual results. Exclude deliberate faults and do not count an execution error as a completed miss. This tests repeatability on cached images and does not establish a production SLO.

4. Which commands restore the workload and prove cleanup? Would restarting alone repair the three injected settings?

Copying startup-good.yaml to startup.yaml removes the injected configuration, and run.sh creates fresh resources for each restored trial. Final Pod and Service gets must both return NotFound; connection errors are inconclusive. A container restart cannot change placement or Service selectors and would repeat the configured 45-second sleep.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Prepare the shell and a reusable diagnostic view: The helper only reads saved JSON; a diagnostic_error must remain missing evidence, not a zero-endpoint result.

Step 2. Run one healthy baseline: A completed healthy trial should report success=1 within 30 seconds, with an assigned Ready Pod and a ready Service endpoint. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 3. Run the 45-second startup-delay fault: The configured 45-second sleep should miss the 30-second budget. The Pod should be assigned to a node but not Ready at capture. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 4. Run the impossible-placement fault: Expect a completed miss with no nodeName and PodScheduled=False / Unschedulable. No application container can start on a node. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 5. Run the wrong-Service-selector fault: Expect a completed miss with an assigned Ready Pod and no matching ready Service endpoint. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 6. Restore the healthy manifest and collect three trials: Each completed healthy run should meet the 30-second target; record actual misses. start.prom contains only the last trial, so it cannot supply the three-trial fraction.

Step 7. Calculate the restored sample success fraction: Three successful restored runs give $3/3 = 1.000$ (100%); otherwise use the observed count. This small controlled sample is not a long-term SLO.

Apply the same reasoning: No. It checks the registry for the image resolution, but cached layers can still be reused. A cold-image experiment must control the node's cache as well.

Lab 13 - Placement, supervision and replacement

1. Compare Pod UID/node and runtime identity before cordon, after cordon, after deletion and after uncordon.

Cordon should set `unschedulable=true` while the same Pod UID and container remain and kubelet stays active. Deleting that Pod should produce a new UID on another eligible node. Uncordon permits future placement but should not move that replacement automatically.

2. Build the kubelet-failure timeline from stop-command completion, Lease, first unavailable/taint observations and replacement UID. Calculate the two observed intervals.

After kubelet stops, Lease `renewTime` should become stale, followed by node-unavailable handling, `NoExecute` taint and possible replacement. Report the measured stop-to-first-unavailable and first-taint-to-first-replacement sample differences. They include sampling uncertainty and separate stages; the 20-second toleration starts at tainting, not at the stop command. Mark absent events not observed.

3. Did the original container keep running while replacement occurred? What proves the difference between API state and actual execution?

If `crictl` still lists `OLD_CID` while the API shows a different Pod UID elsewhere, the old process and replacement coexist. Kubelet reporting and desired API state do not themselves stop a runtime container. A runtime query error is unknown, not evidence that the old process stopped.

4. What evidence confirms recovery, and what remains unknown if no replacement was observed?

After restoring kubelet, all four nodes should become Ready, workers should be uncordoned and node-web should have one Ready Pod. If replacement occurred, `OLD_CID` should disappear as the old node reconciles. If no replacement appeared within the sampling bound, report that limit and still verify kubelet recovery; do not invent an eviction or replacement time.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the original Pod and runtime container: One Ready node-web Pod on a worker; the two `NoExecute` tolerations explicitly say 20 seconds. Save its container ID.

Step 2. Cordon the host and inspect the existing Pod: The same UID and container remain while the node is cordoned; kubelet still runs.

Step 3. Delete the Pod while its node is cordoned: The Deployment should replace the deleted Pod on another eligible node.

Step 4. Uncordon and save a fresh baseline for the kubelet trial: Uncordon should not move the current Pod back. Treat its UID/container as the original identities for the separate kubelet failure.

Step 5. Stop kubelet and save command-completion time: kubelet should report inactive (a nonzero `is-active` exit is expected). The existing application container can remain running.

Step 6. Sample heartbeats, node state and replacement 18 times: The Lease should stop advancing before unavailable handling and eviction. A replacement may appear while `OLD_CID` still runs; if it does not appear within this bound, record not observed and restore kubelet.

Step 7. Restore kubelet and check convergence: Four Ready nodes. If a replacement was created, the old container should disappear as kubelet reconciles. Recheck after a short wait if it is still terminating.

Step 8. Calculate the two observed intervals: These are differences between observation times, not exact transition latencies. A 20-second toleration is only one component of recovery.

Apply the same reasoning: Removing an API object does not fence or stop an unreachable machine. The old process can continue; preventing concurrent writers requires an appropriate fencing or application coordination mechanism.

Lab 14 - Quorum, convergence and application availability

1. Calculate quorum and tolerated failures for 3, 4 and 5 configured voting members.

Quorum is $\text{floor}(n/2)+1$ and tolerated failures are $n - \text{quorum}$: 3 voters need 2 and tolerate 1; 4 need 3 and tolerate 1; 5 need 3 and tolerate 2. Stopping a member does not remove it from configured membership.

2. With one member stopped, compare the write/readback, replica convergence, HTTP and member/leader evidence.

Two surviving voters can commit the one-down value and the desired replica count of three. A completed rollout and Ready count of three separately establish convergence. HTTP can remain available. A leader change is required only if the stopped member was leader or another election occurred; compare recorded member IDs and leader status rather than assuming an election.

3. With two members stopped, compare both write attempts and existing HTTP. What does a client timeout establish?

With one surviving voter, new commits cannot proceed. Record the actual ConfigMap/scale errors and HTTP result separately: existing routing and web processes can keep serving without new etcd writes. A timeout means the caller did not receive an acknowledgement; it is not a general proof that a submitted operation never committed.

4. What did the recovered stored values show before you overwrote them, and what proves the final baseline was restored?

After endpoint health returns, read stored ConfigMap state/resourceVersion and Deployment spec before changing them. The expected unchanged targets are one-down and three replicas, but the observed values resolve uncertainty. Then set recovered and replicas=2 idempotently; successful readback, two Ready replicas, three healthy etcd endpoints and HTTP verify distinct parts of recovery.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Calculate quorum and establish the baseline: Four Ready nodes, 2/2 web pods, HTTP ok, "configmap/quorum-probe patched" and a table with three members, one IS LEADER=true.

Step 2. Stop one member and test writes, HTTP and leader state: Stop exit must be zero and the target runtime list empty before continuing. Two voters retain quorum; the write should succeed after any transient load-balancer/election delay, while HTTP may continue. An unchanged leader is expected when a follower was stopped.

Step 3. Scale with two voters and observe convergence: The scale to three should commit and converge while quorum survives. A successful scale response alone does not prove the replicas are Ready.

Step 4. Stop the second member and compare writes with HTTP: Confirm the second etcd process actually stopped. One of three voters cannot commit new writes; existing HTTP may continue. A client timeout is an unacknowledged outcome that must be checked after recovery.

Step 5. Restore the two members and wait for endpoint health: Continue only with healthy=1. Restoring the original manifests recovers stopped members with intact data; it does not test backup restoration.

Step 6. Read persisted values before restoring configuration: The stored value normally remains one-down and desired replicas remain three, but use actual readback to resolve uncertain responses. Do not overwrite state unless health and both reads succeeded.

Step 7. Restore the explicit baseline and verify all paths: All three endpoints are healthy; the recovered value is readable, desired and Ready replicas are both two, and HTTP works.

Apply the same reasoning: No. Four members require three votes and tolerate one failure, just as three members tolerate one. Five members require three and tolerate two.

Lab 15 - Count how layered retries multiply work

1. For each of the six case/phase rows, record original requests, client attempts, backend arrivals, amplification and successful requests.

Use the six printed phase summaries as your measurements. Each phase submits 20 originals; amplification is $\text{backend_arrivals} / 20$ and success fraction is $\text{successes} / 20$. Check both runner exits are zero before comparing cases.

2. Calculate the maximum backend arrivals during persistent HTTP 503 for zero and one client retry. Compare these bounds with your measurements.

With zero client retries, $20 \text{ originals} \times 1 \text{ client attempt} \times 2 \text{ proxy attempts}$ allows 40 backend arrivals (2x). One client retry allows $20 \times 2 \times 2 = 80$ (4x). Persistent 503 should reach those bounds in this fixture; explain any lower observed count from the logs rather than replacing it with the prediction.

3. Explain why the extra attempts may add no successful requests, why recovery reduces attempts, and what the cleanup checks show.

Both fault cases can finish with zero successes because every backend attempt returns 503. Healthy and recovered phases should need 20 client attempts and 20 backend arrivals for 20 successes because the first success ends retrying. No fail marker or listeners on 8090, 9001 and 9002 confirms the runner removed its fault and processes. This establishes work multiplication, not a self-sustaining overload.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Run the three phases with zero client retries: The runner measures healthy, persistent-503 and recovered phases in order, with 20 originals each. It removes the fault on exit.

Step 2. Change only the client retry count from zero to one: A client retry can double backend work while both upstreams keep failing. Successful phases end retrying after the first success.

Step 3. Compare the six summaries and confirm cleanup: The saved summaries retain both cases. The fault marker is absent and no listeners remain on the three lab ports.

Apply the same reasoning: Three retries means four attempts at each layer, so at most $4 \times 4 = 16$ backend arrivals per original request, provided every attempt reaches the backend and keeps failing.

Lab 16 - Design one clear experiment

1. Name one service and environment. Specify the measurement, units, load, sampling window and exact baseline measurement command.

The completed card should identify one service/environment and an executable measurement with units, load and duration. The baseline command and fault-phase measurement must use the same operation and window so their results can be compared. A statement such as "service is healthy" is insufficient.

2. Specify one fault, its exact target and maximum duration, the injection and confirmation commands, and your predicted measurement threshold.

The card should name one fault and bounded target/duration, provide the exact injection command, and provide a separate command that confirms the target count and dose. The predicted threshold must use the chosen measurement and have a stated mechanism; it remains a prediction until tested.

3. Specify an observable stop threshold, the rollback and fallback commands, and the command and criterion that prove recovery.

The stop threshold states when to abort. Rollback states exactly how to remove the fault; the fallback covers loss of the normal removal path. Recovery requires an executable check against an explicit service criterion, not merely a successful cleanup command.

4. Identify what the plan cannot establish. Review every field for completeness without claiming unmeasured results.

The blank-field check detects omissions only. A reviewer must also confirm command context, values, target scope and measurable criteria. This lab produces a plan; neither a completed card nor an unrun hypothesis establishes resilience.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read the card and choose one service and one fault: Setup creates the card only if it is absent, so an existing plan is preserved. This lab does not execute the proposed fault.

Step 2. Write the complete plan, including executable commands: The saved card should let another operator understand each operation without guessing missing commands or values. Leave observations explicitly unmeasured.

Step 3. Check missing fields and review the saved plan: No blank fields remain. The check does not validate meaning or execute commands, so confirm the measurement, fault boundaries and recovery steps by reading the card.

Apply the same reasoning: No. The card is a plan, and the control establishes the measurement path and baseline. Only an observed fault trial and analysis can support the specific resilience claim.

Lab 17 - Signal delivery and the shutdown budget

1. Compare the wrapper and exec cases with the same five-second stop budget. What worker PID, TERM receipt, cleanup marker, exit code and OOM flag did each produce?

The wrapper case should report a worker PID other than 1, no received=15 line and no cleanup marker: the wrapper ignores TERM and does not forward it. The exec/five-second case should report PID 1, received=15, a completion line/file and exit zero. Cite the actual logs and Docker state for each case.

2. Compare the two exec cases. Which evidence separates successful signal delivery from enough time to finish cleanup?

Both exec cases should receive TERM. Five seconds accommodates the two-second cleanup; one second should allow the receipt/start log but prevent the completion marker. Correct signal delivery and sufficient cleanup time are separate requirements.

3. Explain the cause of each forced exit and show that the test containers were removed after recording their evidence.

For the wrapper and short-budget cases, exit 137 with OOMKilled=false and the controlled docker stop operation is consistent with a forced shutdown after the grace budget. Exit 137 alone is not an OOM diagnosis. The final container-list command should be empty after evidence collection and removal.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Start the wrapper case and confirm the worker is ready: The wrapper is PID 1 and the Python worker is its child, so the worker PID is not 1.

Step 2. Stop the wrapper with five seconds and collect the result: The child should receive no TERM, create no marker and be forcibly stopped after the grace budget. A missing marker-copy source is expected in this case.

Step 3. Start the exec case with the same application: Exec makes the worker PID 1 so Docker can deliver its stop signal directly.

Step 4. Stop exec with five seconds and collect the result: The worker should receive TERM, finish its two-second cleanup, write completed and exit zero.

Step 5. Start another exec case for the shorter budget: The worker is PID 1 again; the next stop changes only the time budget.

Step 6. Stop exec with one second and collect the result: TERM should reach the worker, but the one-second budget is shorter than its cleanup. A missing marker-copy source is expected.

Step 7. Remove the three stopped containers after recording evidence: No ce-lab17 containers remain; the successfully copied exec-complete.txt file remains in the VM workspace.

Apply the same reasoning: No. More time cannot make this wrapper forward a signal it ignores. Repair the process relationship or implement correct forwarding first.

Lab 18 - Persistence, ownership and read-only mounts

1. Which file survives container replacement? Use the original and replacement observations to distinguish a named volume from the container writable layer.

The original container creates /ephemeral.txt in its writable layer and /data/persisted.txt in the named volume. After that container is removed, the replacement should read persisted.txt but find no ephemeral.txt. The named volume has a separate lifetime; this observation does not establish a backup.

2. Why does UID 10001 initially fail to write, and which exact ownership change repairs it? Cite numeric ownership, mode and write status.

Initially /data is owned by UID 0 with mode 0755, so UID 10001 lacks directory write permission. chown 10001:10001 /data assigns only the lab directory to the intended identity. The repeated write should then return zero without changing its mode or granting world write access.

3. Why does the same user fail through the read-only mount? Compare the error and RW field, then prove writable recovery and data persistence.

With ownership unchanged, a read-only mount should fail with Read-only file system and RW=false. Restoring a writable mount should produce RW=true, a successful write and readable recovered/volume contents. This separates mount enforcement from Unix ownership and from data loss. Record those results before deleting the dedicated test volume.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Create one file in each storage layer: Both files exist in the original container; only /data is backed by ce-lab18-data.

Step 2. Replace the container and check which file remains: The volume file survives replacement while the removed container's writable-layer file does not.

Step 3. Attempt the write as UID 10001 and diagnose the failure: UID 10001 should get Permission denied because UID 0 owns the directory and mode 0755 permits only the owner to create files.

Step 4. Change only the directory owner and repeat the write: The write should succeed as the new owner; no chmod 777 is needed.

Step 5. Change only the mount to read-only and inspect the failure: Correct ownership cannot override RW=false; the write should fail with Read-only file system.

Step 6. Restore a writable mount and confirm both data and writes: The write should succeed with RW=true, and the output should contain recovered plus the original volume content.

Step 7. Remove the exercise containers and volume after recording results: The cleanup removes only the dedicated exercise storage. original-layer.txt remains outside the volume as saved evidence.

Apply the same reasoning: No. Mount read-only enforcement is independent of Unix mode bits. For an ownership mismatch, inspect the required identity and grant only the needed access.

Lab 19 - Unscheduled Pods versus memory-killed containers

1. Compare the healthy Pod with the 1000-CPU request: which stage fails, and what do node assignment, PodScheduled, current-UID events and logs show?

The healthy fixture should be node-assigned and Ready with a 20m CPU request. The 1000-CPU request should remain unassigned with PodScheduled=False/Unschedulable and current-UID scheduling events showing insufficient CPU on eligible workers. Its application has not started, so a failed logs request is not an application crash.

2. For the 32 MiB memory limit, record the request, deliberate allocation, node, UID, termination reason, exit and restart count. How does this differ from the scheduling failure?

The OOM fixture retains a fitting 16 MiB memory request but has a 32 MiB limit and deliberately allocates 96 MiB. It should be assigned to a node, then show OOMKilled in the previous termination, typically exit 137 and a positive restart count. The previous log should identify the allocation. Use OOMKilled and the resource evidence, not exit 137 or CrashLoopBackOff alone, to diagnose the runtime failure.

3. Show that the restored Pod is Ready with the healthy resources. Explain which UID changes were intentional replacement and which restart happened within one Pod.

Applying good.yaml after deleting the faulty Pod should produce a new Ready Pod with the healthy resource configuration and no previous OOM/repeated restarts. The command sequence deliberately changes UID between fixtures. Compare the OOM UID captured at creation with the crash snapshot to show that kubelet restarted a container inside the same Pod.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the healthy placement and configured resources: The baseline should be assigned and Ready. Requests describe scheduling demand; limits describe runtime bounds.

Step 2. Replace the baseline with the impossible CPU request: The request is 1000 CPUs, not 1000m. No eligible worker can satisfy it, so the Pod should remain unassigned.

Step 3. Collect scheduling events and check application-log availability: Events should explain failed placement. The logs request should fail because there is no started application container.

Step 4. Replace it with the memory failure and wait for OOM evidence: This request fits, but the deliberately oversized allocation should exceed the runtime memory limit.

Step 5. Save the runtime crash, prior logs and current-UID events: The previous instance should be OOMKilled inside the same Pod UID. Capture the reason and log while the Pod still exists; rerun this observation if restart status is updating.

Step 6. Restore the healthy fixture and verify recovery: The new healthy Pod should be Ready with no OOM termination or repeated restarts. Saved fault evidence remains in the lab workspace.

Apply the same reasoning: No. Placement uses the request and available allocatable capacity. Increasing a limit does not reduce that request or add a suitable node.

Lab 20 - Trace DNS, Service ports and the application listener

1. Record the healthy DNS, Service-name HTTP, Service-IP HTTP, direct-Pod HTTP, readiness and endpoint port. Which of these paths does each probe test?

The healthy fixture should resolve the Service name and return HTTP 200 through the name, ClusterIP:80 and PodIP:8080, with a Ready endpoint at 8080. DNS tests naming; direct Pod HTTP bypasses Service routing; Service HTTP includes the Service path. Record actual results for each path.

2. For the wrong namespace name, which requests fail and which still work? Prove recovery by using the correct name.

The missing namespace name should fail resolution and name-based HTTP while the correct ClusterIP and correct name still work. Returning to the correct name restores the request without changing the DNS server or the cluster.

3. For the wrong targetPort, compare DNS, Service and direct-Pod results with the endpoint port. Show the result after restoring 8080.

With targetPort 8099, DNS and direct PodIP:8080 HTTP should still work, while Service traffic goes to an unused endpoint port and fails. A Ready Pod or a nonempty EndpointSlice does not prove that the selected port is correct. Restoring targetPort 8080 should restore Service HTTP.

4. For the loopback-only listener, compare localhost, remote Pod-IP and Service HTTP with readiness. Show the full path working after listener recovery.

A server bound to 127.0.0.1 should answer inside its own Pod but reject access to its Pod IP from the client. Its HTTP readiness probe uses the Pod IP and should fail, making the endpoint unready. Recreating it with a 0.0.0.0 listener should restore readiness, endpoint eligibility and DNS/Service/direct-Pod HTTP. Local success alone does not prove remote reachability.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Load the client helpers and bounded endpoint-port check: Sourcing the helpers targets only Lab 20. The port check succeeds only when a nonempty current slice list uses the requested port.

Step 2. Measure the healthy request path from the client Pod: The correct name resolves, all three HTTP paths should succeed, and the ready endpoint should use port 8080.

Step 3. Request the wrong namespace name, then retry the correct path: The incorrect name should fail while the correct IP and name work, isolating the naming error.

Step 4. Change targetPort to 8099 and compare Service with direct HTTP: DNS and direct Pod HTTP should work, but Service traffic should fail because it forwards to unused port 8099.

Step 5. Restore targetPort to 8080 before the next fault: Restoring 8080 should repair the Service path; allow a short bounded interval for the routing update.

Step 6. Replace only the server bind address and compare local with remote: Local HTTP should succeed while remote Pod HTTP fails and readiness is false. DNS can continue working even though the Service has no ready backend.

Step 7. Restore the listener and check every path again: The 0.0.0.0 listener should restore HTTP from other Pods, a Ready endpoint at 8080 and successful Service requests.

Apply the same reasoning: Not from that evidence. The direct request bypasses the Service selector. Investigate the process listener, destination port and Pod network path first.

Lab 21 - Configuration freshness and a stalled rollout

1. Compare the ConfigMap, Pod UIDs and per-Pod HTTP responses before and after changing only the ConfigMap. Did the running processes consume the new value?

Changing only the ConfigMap to version-two should leave the existing Pod UIDs and their version-one HTTP responses unchanged. The message entered each process through its startup environment; updating the separate ConfigMap does not refresh that environment or change the Pod template.

2. After rollout restart, record the UIDs, responses and stable template revision. What operation made configuration take effect?

rollout restart changes the Pod template and replaces consumers. The new UIDs should return version-two after completion. Save the resulting revision before the fault so undo selects the known-good readiness template; the revision identifies a template, not a ConfigMap snapshot.

3. For the bad readiness port, explain the new Pod state, old-replica availability, replica counts and Progressing condition. Does direct application HTTP still work?

With readiness port 9999, the surge Pod should run but remain unready while its process can still answer on 8080. maxUnavailable=0 keeps the two old replicas available; maxSurge=1 limits added replicas. The stalled update should eventually report ProgressDeadlineExceeded. Record actual counts, conditions and restart counts; a readiness failure does not itself imply a crash or automatic rollback.

4. After undo, what did the template restore and what configuration stayed changed? Show the separate actions and evidence that return every consumer to version-one.

Undo to the saved revision should restore readiness port 8080 and complete the rollout, while the ConfigMap and consumer responses remain version-two. Explicitly changing the ConfigMap back to version-one and restarting consumers should produce two Ready consumers returning version-one. Verify the ConfigMap, probe port, replica counts and every response separately.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the original configuration and every consumer: The baseline should have two Ready consumers returning version-one. responses queries each current Pod directly on port 8080.

Step 2. Change only the ConfigMap and query the same processes: The ConfigMap should say version-two while unchanged consumers still return version-one; this edit alone does not start a rollout.

Step 3. Restart consumers and save the good revision for undo: New consumers should read version-two. The good probe remains 8080 and the completed rollout should have two Ready current consumers; responses excludes terminating Pods.

Step 4. Break the new readiness port and collect stalled-rollout evidence: The new Pod should be unready at port 9999 while two old replicas remain available. Its process can still answer direct HTTP on 8080. ProgressDeadlineExceeded reports the stall without undoing it.

Step 5. Undo to the saved template and inspect configuration separately: The good readiness port should return to 8080, while the ConfigMap and running responses remain version-two.

Step 6. Restore version-one and verify configuration plus all consumers: The ConfigMap should say version-one, readiness should use 8080, and two Ready current consumers should return version-one after a successful rollout.

Apply the same reasoning: No. That revision still refers to the same ConfigMap key. Newly created containers read its current value; the ConfigMap must be restored separately or configurations versioned under distinct names.